

ABSTRACT

Indoor navigation has always been a challenge for visually impaired (VI) individuals as indoor environments are more complex and clustered than outdoor ones. In this paper, a smartphone-based QR code indoor navigation system is proposed to assist VI individuals navigate indoor environments independently and safely. Various techniques are used to detect QR codes and perform pathfinding. However, the main priority is to guide VI individuals to their destinations safely and efficiently.

The proposed system uses OpenCV for colour processing to extract coloured QR codes in red, blue, or green, making them easier for VI users to identify. A fine-tuned YOLOv5 object detection model is then applied to accurately detect and scan QR codes containing location data. Once scanned, the system determines the user's current position and computes the shortest route to the destination using the A* algorithm. For ease of use, a simple user interface delivers real-time and step-by-step navigation instructions through voice guidance.

TABLE OF CONTENTS

TITLE PAGE	i
ABSTRACT	ii
TABLE OF CONTENTS	iii
LIST OF FIGURES	vi
LIST OF TABLES	ix
LIST OF SYMBOLS	x
LIST OF ABBREVIATIONS	x

CHAPTER 1 INTRODUCTION	1
1.1 Problem Statement and Motivation	1
1.2 Project Scope, Objectives	1
1.3 Impact, Significance and Contribution	2
1.4 Background Information	3
CHAPTER 2 LITERATURE REVIEW	4
2.1 Review of Survey Literature on Vision-based Indoor Navigation	4
2.2 Colour Processing for QR Code Isolation in Indoor Environments	6
2.3 Review of QR Code Detection Algorithms	7
2.4 Review of Technologies and Approaches in Indoor Navigation	9
2.5 Comparison of Methods	12
2.5.1 Marker vs Marker-less Approach	12
2.5.2 Comparison of Colour Processing for QR Code Techniques	13
2.5.3 Comparison of QR Code Detection Techniques	14
2.5.4 Comparison of Navigation Techniques	14

CHAPTER 3 PROPOSED METHOD/APPROACH	15
3.1 Design Specification	15
3.1.1 Methodologies and General Work Procedures	15
3.1.2 Tools to Use	16
3.1.3 User Requirements	16
3.1.4 System Performance Definition	17
3.1.5 Verification Plan	17
3.2 System Design Diagram/Equation	18
3.2.1 Destination Input	19
3.2.2 QR Code Detection	19
3.2.2.1 Bluetooth-based Area Reduction	19
3.2.2.2 Colour Segmentation and Contour Detection	19
3.2.2.3 YOLOv5 QR Code Detector Model	19
3.2.3 Initialisation and Path Planning	20
3.2.3.1 QR Code Database	20
3.2.3.2 Path Planning and Route Optimisation	20
3.2.3.3 Cognitive Navigation Cluttered Spaces	20
3.2.4 Navigation	20
3.2.4.1 Voice and Haptic Guidance	20
3.2.4.2 Integration with other sensors for localisation	21
3.2.4.3 Deviation Detection Module	21
3.2.4.4 Obstacle Detection and Avoidance Model	21
3.2.4.5 Destination Reached Signal	21
3.3 Implementation Issues and Challenges	21
3.4 Timeline and Milestone for 14 Weeks	22
 CHAPTER 4 PRELIMINARY WORK	 24
4.1 Preliminary Work 1	24
4.2 Preliminary Work 2	24
4.3 Preliminary Work 3	26
4.4 Preliminary Work 4	27

CHAPTER 5 CONCLUSION	28
5.1 Summarisation of Finding	28
5.2 Novelties of Work	29
5.3 Concluding with Supportive Remark	30
 BIBLIOGRAPHY/REFERENCES	 31
 APPENDIX A	
A.1 Preliminary Work 1 Codes	A-1
A.2 Preliminary Work 2 Codes	A-4
A.3 Preliminary Work 3 Codes	A-6
A.4 Preliminary Work 4 Codes	A-12
 APPENDIX B	
B.1 Preliminary Work 1 Coding Results	B-1
B.2 Preliminary Work 2 Coding Results	B-2
B.3 Preliminary Work 3 Coding Results	B-12
B.4 Preliminary Work 4 Coding Results	B-14
 APPENDIX C	
Plagiarism Check Result	C-1

LIST OF FIGURES

Figure Number	Title	Page
Figure 2.1.1	Breakdown of Indoor Navigation Systems [3]	4
Figure 2.1.2	Classification of Methods and Systems in Vision-Based Approaches [4]	5
Figure 2.2.1	Process of K-means Clustering in [7]	6
Figure 2.2.2	Process of Region Growing in [6]	7
Figure 2.3.1	QR Code Detection Process of [8]	8
Figure 2.3.2	QR Code Recognition Process in [9]	9
Figure 2.3.3	Image Preprocessing Process in [9]	9
Figure 2.4.1	Navigation with Real-time Obstacle Avoidance [10]	10
Figure 2.4.2	Navigation System Functional Block Diagram [10]	10
Figure 2.4.3	Architecture of The Navigation System [11]	11
Figure 3.1.1.1	System Flowchart for General Work Procedures	15
Figure 3.2.1	System Overview Block Diagram	18
Figure 3.4.1	Milestones Chart of The Project	22
Figure 3.4.2	Gantt Chart for Project Timeline	23
Figure 4.3.1	QR Image Processing Steps before Decoding	26
Figure B.1.1	Selecting QR Code Colour	B-1
Figure B.1.2	Confirming QR Code Colour	B-1
Figure B.1.3	QR Code Detected	B-1
Figure B.1.4	Guiding User Towards the QR Code	B-1
Figure B.1.5	QR Code Has Been Read	B-1
Figure B.1.6	Wrong QR Code Colour	B-2
Figure B.2.1	QR Code Detection at 50 cm (81 cm ² , Low Light)	B-2
Figure B.2.2	QR Code Detection at 100 cm (81 cm ² , Low Light)	B-2
Figure B.2.3	QR Code Detection at 150 cm (81 cm ² , Low Light)	B-3
Figure B.2.4	QR Code Detection at 50 cm (Red Module + White Background, 324 cm ² , Low Light)	B-3
Figure B.2.5	QR Code Detection at 50 cm (Red Module + White Background, 324 cm ² , Low Light)	B-3

Figure B.2.6	QR Code Detection at 50 cm (Red Module + White Background, 324 cm ² , Low Light)	B-4
Figure B.2.7	QR Code Detection at 50 cm (White Module + Red Background, 324 cm ² , Low Light)	B-4
Figure B.2.8	QR Code Detection at 100 cm (White Module + Red Background, 324 cm ² , Low Light)	B-4
Figure B.2.9	QR Code Detection at 150 cm (White Module + Red Background, 324 cm ² , Low Light)	B-5
Figure B.2.10	QR Code Detection at 50 cm (Red Module + Black Background, 324 cm ² , Low Light)	B-5
Figure B.2.11	QR Code Detection at 100 cm (Red Module + Black Background, 324 cm ² , Low Light)	B-5
Figure B.2.12	QR Code Detection at 150 cm (Red Module + Black Background, 324 cm ² , Low Light)	B-6
Figure B.2.13	QR Code Detection at 50 cm (Black Module + Red Background, 324 cm ² , Low Light)	B-6
Figure B.2.14	QR Code Detection at 100 cm (Black module + Red Background, 324 cm ² , Low Light)	B-6
Figure B.2.15	QR Code Detection at 150 cm (Black module + Red Background, 324 cm ² , Low Light)	B-7
Figure B.2.16	QR Code Detection at 50 cm (81 cm ² , Bright)	B-7
Figure B.2.17	QR Code Detection at 100 cm (81 cm ² , Bright)	B-7
Figure B.2.18	QR Code Detection at 150 cm (81 cm ² , Bright)	B-8
Figure B.2.19	QR Code Detection at 50 cm (Red Module + White Background, 324 cm ² , Bright)	B-8
Figure B.2.20	QR Code Detection at 50 cm (Red Module + White Background, 324 cm ² , Bright)	B-8
Figure B.2.21	QR Code Detection at 50 cm (Red Module + White Background, 324 cm ² , Bright)	B-9
Figure B.2.22	QR Code Detection at 50 cm (White Module + Red Background, 324 cm ² , Bright)	B-9
Figure B.2.23	QR Code Detection at 100 cm (White Module + Red Background, 324 cm ² , Bright)	B-9

Figure B.2.24	QR Code Detection at 150 cm (White Module + Red Background, 324 cm ² , Bright)	B-10
Figure B.2.25	QR Code Detection at 50 cm (Red Module + Black Background, 324 cm ² , Bright)	B-10
Figure B.2.26	QR Code Detection at 100 cm (Red Module + Black Background, 324 cm ² , Bright)	B-10
Figure B.2.27	QR Code Detection at 150 cm (Red Module + Black Background, 324 cm ² , Bright)	B-11
Figure B.2.28	QR Code Detection at 50 cm (Black Module + Red Background, 324 cm ² , Bright)	B-11
Figure B.2.29	QR Code Detection at 100 cm (Black Module + Red Background, 324 cm ² , Bright)	B-11
Figure B.2.30	QR Code Detection at 150 cm (Black Module + Red Background, 324 cm ² , Bright)	B-12
Figure B.3.1	Decoding Success Rates of ZBar and OpenCV	B-12
Figure B.3.2	Examples of Successfully Decoded QR Code Images	B-12
Figure B.3.3	Example of Failed Decoding Using Binarized Images	B-13
Figure B.3.4	Examples of Failed Decoding due to Perspective Correction Failure and Image Blur	B-13
Figure B.3.5	Decoding Counts for Binarized, Upscaled, and Grayscale Images	B-13
Figure B.3.6	Examples of Failed Decoding due to Lighting Reflection	B-14
Figure B.3.7	Examples of Failed Decoding of Distorted QR Code Images	B-14
Figure B.3.8	Examples of Failed Decoding due to Shadow	B-14
Figure B.4.1	The Output of Successfully Generating the UTAR FICT Layout	B-15
Figure B.4.2	The Adjacent and Neighbouring Objects for each Object	B-15
Figure B.4.3	Pathfinding Result of A* from N001 to All Available Destinations	B-15
Figure B.4.4	A Portion of The UTAR FICT Ground Floor Map with The Starting Point at N001	B-16

LIST OF TABLES

Table Number	Title	Page
Table 2.5.1	Comparison between Marker-Based and Markerless Approach	12
Table 2.5.2	Comparison of Colour Processing Techniques	12
Table 2.5.3	Comparison of QR Code Detection Techniques	13
Table 2.5.4	Comparison of Navigation Techniques	14
Table 3.1.2.1	Hardware Specifications	16
Table 4.2.1	QR Code Detection Result under Various Conditions	25

LIST OF SYMBOLS

A algorithm* A star algorithm

LIST OF ABBREVIATIONS

<i>VI</i>	Visually Impaired
<i>QR</i>	Quick Response
<i>YOLO</i>	You Only Look Once
<i>A* algorithm</i>	A star algorithm
<i>GPS</i>	Global Positioning System
<i>UI</i>	User Interface
<i>AR</i>	Augmented Reality
<i>WiFi</i>	Wireless Fidelity
<i>IMU</i>	Inertial Measurement Unit
<i>2D</i>	Two Dimensions
<i>3D</i>	Three Dimensions
<i>SLAM</i>	Simultaneous Localization and Mapping
<i>SfM</i>	Structure from Motion
<i>CNN</i>	Convolution Neural Networks
<i>HSV</i>	Hue, Saturation, and Value
<i>RGB</i>	Red, Green, Blue
<i>HS</i>	Hue-Saturation
<i>HV</i>	Hue-Value
<i>SV</i>	Saturation-Value
<i>BP</i>	Backpropagation neural network
<i>CAD</i>	Computer-Aided Design
<i>RGB-D</i>	Red, Green, Blue Depth
<i>TSM</i>	Time-Stamped Map
<i>SVD</i>	Singular Value Decomposition
<i>ADF</i>	Area Description File
<i>KF</i>	Kalman Filter
<i>HMI</i>	Human-Machine Interface
<i>SURF</i>	Speeded-Up Robust Features

<i>AI</i>	Artificial Intelligence
<i>OSTU algorithm</i>	Ōtsu Nobuyuki
<i>OS</i>	Operating System
<i>CPU</i>	Central Processing Unit
<i>GPU</i>	Graphics Processing Unit
<i>RAM</i>	Random-Access Memory
<i>RTX</i>	Ray Tracing Texel
<i>GDDR6</i>	Graphics Double Data Rate 6
<i>DDR5</i>	Double Data Rate 5
<i>SSD</i>	Semiconductor Storage Device
<i>RSSI</i>	Received Signal Strength Indicator
<i>UTAR</i>	University Tunku Abdul Rahman
<i>FICT</i>	Faculty of Information and Communication Technology

CHAPTER 1

Introduction

1.1 Problem Statement and Motivation

Individuals with visual impairments often struggle with indoor navigation. Although outdoor navigation technologies such as the Global Positioning System (GPS) are well-established, they are not suitable for indoor environments due to weak signal reception. According to [1], indoor navigation remains a significant challenge for visually impaired (VI) individuals because of the lack of a reliable and accessible wayfinding solution in a clustered indoor environment. Existing indoor navigation systems commonly have drawbacks, such as being too costly, lacking real-time updates and dynamic obstacle detection, or not providing intuitive guidance tailored to their needs [2].

Therefore, this project aims to develop an advanced and affordable vision-based system to assist visually impaired individuals with indoor navigation. By leveraging computer vision and the smartphone's built-in camera, the system enables accurate positioning and guidance. The successful implementation of this solution would enhance safety, improve accessibility, promote greater independence, and reduce reliance on smart canes for guiding visually impaired individuals in indoor environments.

1.2 Project Scope and Objectives

Project Scope

This project focuses on developing an indoor navigation system for the visually impaired (VI) within the Faculty of Information and Communication Technology (FICT) building at Universiti Tunku Abdul Rahman (UTAR). This project uses coloured QR codes as location markers, which are placed on door glass panels throughout the building. VI will be directed to the nearest QR code location to scan the QR code with a camera-enabled device.

CHAPTER 1

The current location of VI will be determined, and navigation guidance will be provided based on the shortest path calculated for VI to reach the destination.

The scope of this project is limited to the ground floor of the FICT building, assuming that pathways are free of obstacles. The project will not cover outdoor navigation, GPS integration, real-time obstacle detection, or deployment in other buildings.

Project Objectives

- i. To develop an indoor navigation system for the visually impaired (VI) using coloured QR codes**

VI individuals navigate indoor environments by scanning QR codes placed on the door-glass. The system will decode each QR code location information to determine the VI's current position. With this information, the user is guided to their desired destination.

- ii. To integrate the model, algorithm, and APIs for better navigation**

This involves combining different technological components to enhance the navigation experience. A fine-tuned object detection model, YOLOv5, is used to detect QR codes quickly and accurately. Moreover, the A* algorithm, which is a pathfinding algorithm, is integrated to calculate the shortest and most accessible route. On the other hand, services or libraries for voice guidance, text-to-speech, or mapping functionalities will be implemented to ensure smooth operation from QR code detection to delivering clear navigation instructions.

1.3 Impact, Significance and Contribution

This study addresses navigation challenges encountered by the visually impaired (VI), for whom moving around indoors in an unfamiliar environment without visual cues can be challenging. This often results in heavy reliance on others and potential safety hazards. By implementing a robust and inexpensive indoor navigation system using QR codes, VI users can receive real-time, step-by-step audio and haptic instructions, allowing them to reach their

CHAPTER 1

destinations independently and safely. This approach enhances personal safety, mobility, and promotes equal access to public spaces for all members of society.

This approach is important because it provides an affordable indoor navigation solution. Traditional solutions often require expensive and sophisticated methodologies which are not easily adoptable. Our QR code-based solution addresses this shortcoming by leveraging the ubiquity of smartphone technology and a simple, cost-effective marker system.

Our key contribution lies in the development of a fully integrated, user-centric system. This includes prioritising intuitive voice and haptic feedback, a simple user interface, and accessible route planning to transform indoor environments into spaces where VI individuals can navigate freely and confidently.

1.4 Background Information

Indoor navigation has garnered attention in recent years due to its broad applications in surveillance, robotics, Augmented Reality (AR), and assistive navigation systems [3, 4]. This surge in interest is driven by the increasing ubiquity of smartphones and other wireless devices [5]. According to [5], indoor localisation refers to determining the position of a device or individual within an enclosed environment. Although the GPS excels in outdoor navigation, it often fails in indoor navigation [4]. This limitation poses significant challenges for visually impaired (VI) individuals, who rely on accurate and accessible guidance to navigate unfamiliar spaces safely. To overcome the limitations of GPS indoors, solutions such as wireless technologies – WiFi, Bluetooth beacons, inertial sensors, and computer vision have been employed [3, 4].

CHAPTER 2: Literature Reviews

2.1 Review of Survey Literature on Vision-based Indoor Navigation

Vision-based solutions stand out for their precision, affordability, and compatibility with modern devices that already feature advanced cameras, further decreasing the costs and power consumption [3, 4]. Additionally, the chances of information delay are minimised [4]. These systems also provide the flexibility to integrate other technologies like Inertia Measurement Unit (IMU) and wireless signals, increasing accuracy [3, 4]. However, vision-based indoor navigation systems commonly face challenges: maintaining accuracy in varying lighting and managing object occlusions [3, 4]. Cameras, regardless of whether they are 2D or 3D, can struggle with untextured or reflective surfaces, affecting detection and depth estimation. Furthermore, balancing high computational costs with feature complexity and ensuring user comfort remains a persistent hurdle [3].

According to Figure 2.1.1, vision-based approaches are broadly categorised into marker-based and markerless methods. These systems are further classified by platform or application, such as robot navigation systems, wearable technology, AR visualization systems, and mobile device-based systems [4].

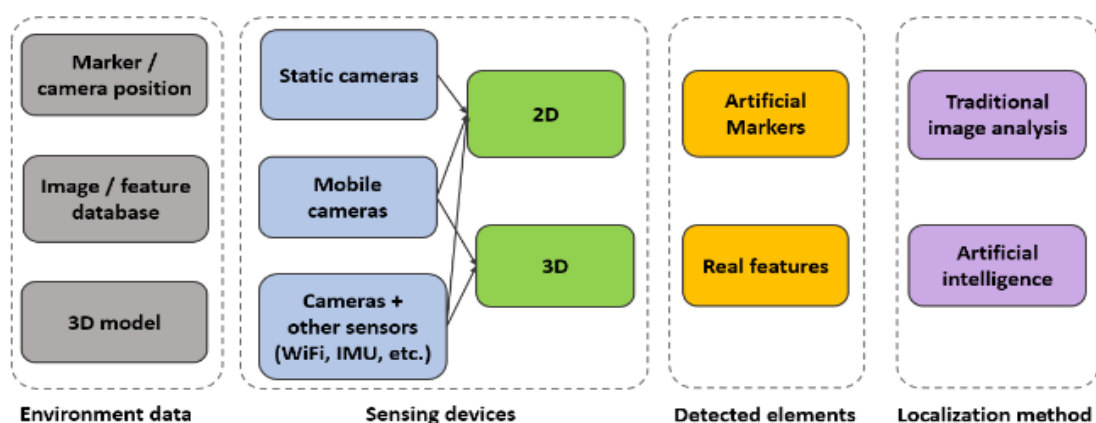


Figure 2.1.1: Breakdown of Indoor Navigation Systems [3]

CHAPTER 2

Based on Figure 2.1.2, all indoor positioning methods begin with an initial configuration stage to collect environmental data, such as physical marker positions, static camera locations, annotated image databases, or 3D models (via manual modeling, SLAM, or SfM). These methods are further distinguished by sensing devices —typically static or mobile 2D/3D cameras, potentially supplemented by WiFi/IMU data — and by their reliance on artificial markers or natural environmental features. Localisation techniques range from traditional image analysis to AI-based approaches [3]. Traditional methods detect features by comparison, while AI techniques, such as convolutional neural networks (CNNs), learn features directly from images, reducing the need for manual selection. However, AI approaches often require large training datasets, which can be a limitation due to the scarcity of publicly available 3D datasets [3]. These classifications support systematic analysis of diverse implementations and help identify common challenges and opportunities across application contexts [3, 4].

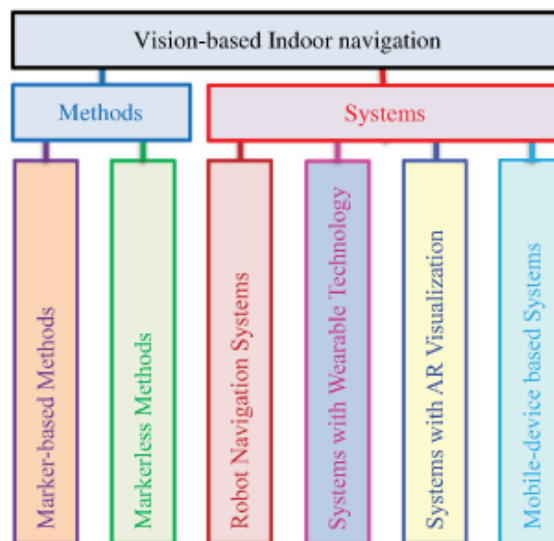


Figure 2.1.2: Classification of Methods and Systems in Vision-Based Approaches [4]

Vision-based systems often rely on handheld or wearable devices. These devices aid by providing audio and haptic feedback to the user. However, such implementations may impact user comfort and increase costs [4]. Marker-based and markerless methods vary in their advantages and disadvantages, as discussed in Section 2.5.

2.2 Colour Processing for QR Code Isolation in Indoor Environments

A key consideration for colour-based detection is the choice of colour space. In [6], the HSV colour space was utilised for its ability to separate brightness intensity from hue and saturation, making it more robust for colour-based segmentation compared to RGB. The target colour distribution was modelled with Gaussian Mixture Models trained on sample images. To reduce computational complexity, the 3D HSV colour distribution is projected onto the Hue-Saturation (HS), Hue-Value (HV), and Saturation-Value (SV) planes. Pixels within these predefined thresholds on these three planes were segmented as fire regions. Although their approach is originally applied to fire detection, the same principle can be adapted for QR code detection, as it can be utilised to recognise the colour of QR codes. However, a key limitation is the need for a large training dataset to capture colour variations accurately and the high computational cost during training and modelling.

In addition, [7] discussed similarity-based segmentation, which includes the k-means clustering technique and the region growing technique. K-means clustering technique groups pixels based on colour similarity by pushing each pixel value towards cluster centres and iteratively updating the mean of assigned pixels until convergence, as shown in Figure 2.2.1. However, k-means clustering depends heavily on the initial placement of cluster centres, which often requires post-processing to address noise and over-segmentation and is prone to slow convergence. These drawbacks can result in inconsistent segmentation and potential latency, making it unsuitable for real-time systems.

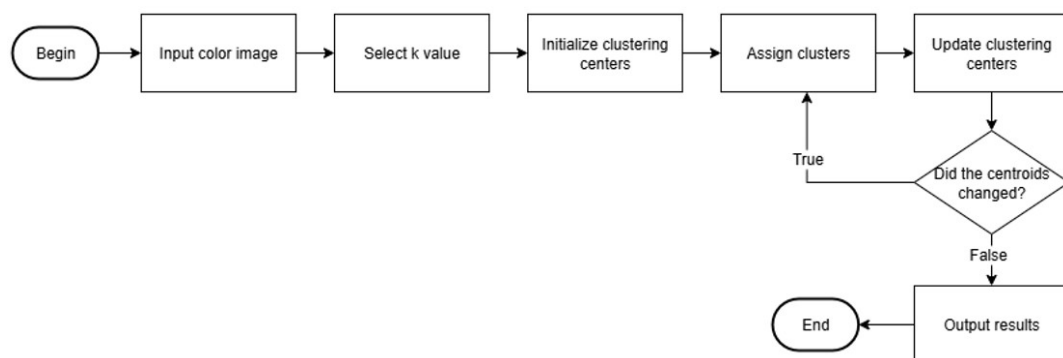


Figure 2.2.1: Process of K-means Clustering in [7]

The region growing method groups neighbouring pixels into regions based on homogeneity criteria such as colour similarity and spatial proximity [7]. This method can generate smooth, contiguous segmentation masks, making it effective for isolating the target colour of the QR code. Based on Figure 2.2.2, the process begins from an initial seed point and expands by adding adjacent pixels that meet the predefined criteria. Despite this advantage, this method is highly sensitive to seed placement and homogeneity parameters, as an inappropriate seed can cause inaccurate segmentation.

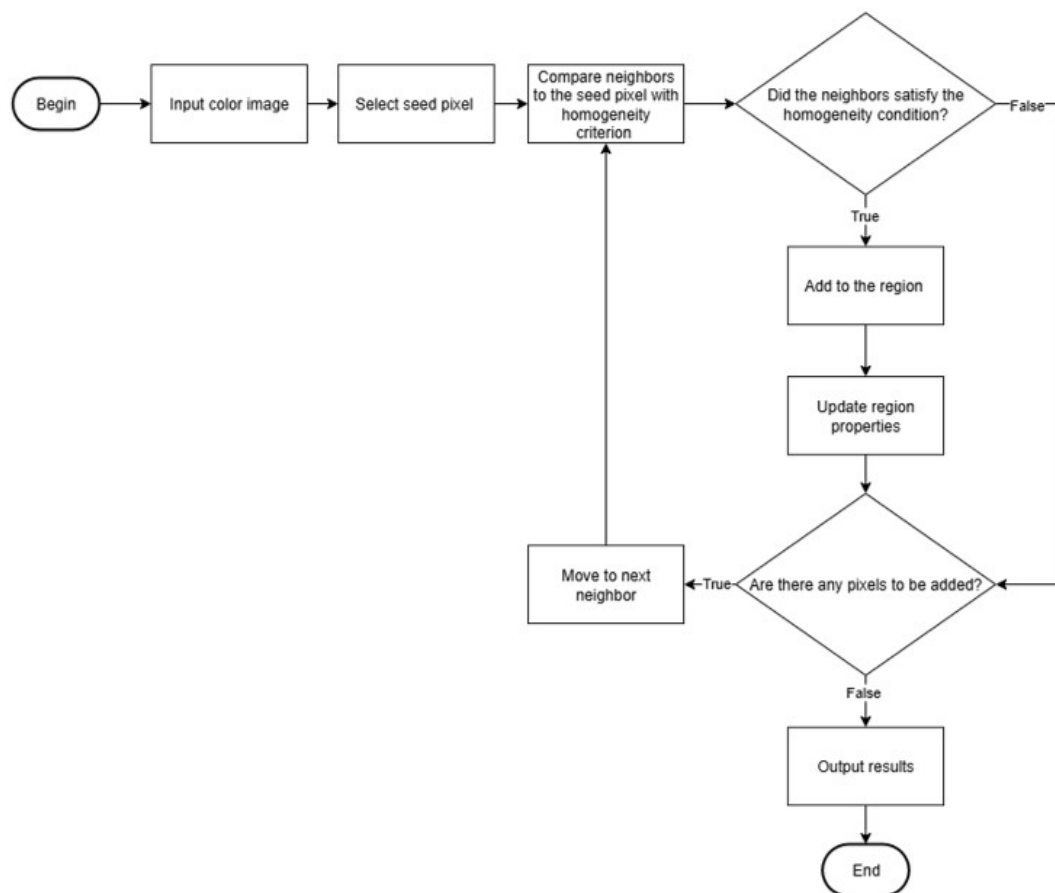


Figure 2.2.2: Process of Region Growing in [6]

2.3 Review of QR Code Detection Algorithms

The paper [8] developed an indoor localization and navigation system for mobile robots using QR codes. Although originally built for robots, their implementation can be adapted for

a VI navigation system. The authors placed the QR codes at fixed points in a grid-based interior area with each QR containing its location coordinates, with detection illustrated in Figure 2.3.1.

The camera captures each step of the robot's movement and transmits the images to a connected computer. The pictures are then transformed into gray-scale images to identify the shape of the QR code using feature extraction. The detected QR codes are decoded with ZBar and Open Source Computer Vision (OpenCV), establishing the current position of the robot. A Dijkstra algorithm is applied to determine the shortest path to a target destination. Once a path has been determined, the robot would monitor its surroundings as it navigates, searching for QR codes continuously.

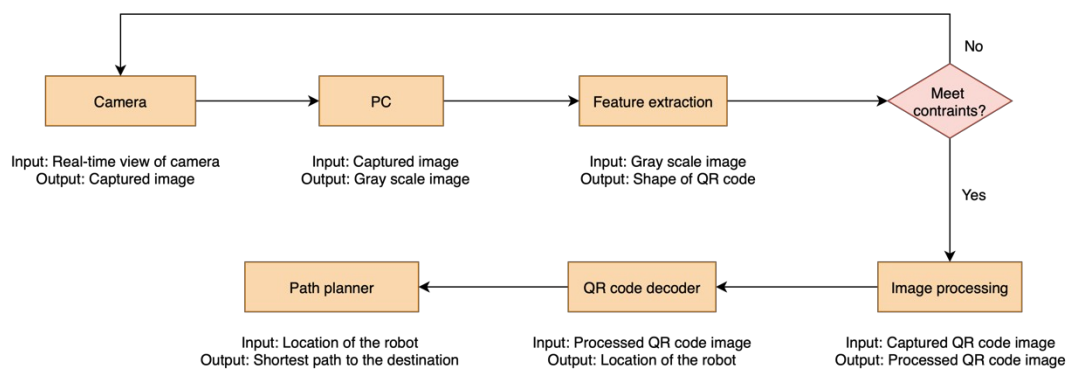


Figure 2.3.1 QR Code Detection Process of [8]

Huo et al. [9] suggested an artificial intelligence algorithm-based system for the recognition of QR image code. Their solutions consider issues faced by QR code recognition modules, such as background noise, poor lighting, and distortion. As illustrated in Figure 2.3.2 and Figure 2.3.3, the process begins with image capture, followed by grayscale transformation to reduce colour space and maximize contrast. Further image processing is done with an improved filtering algorithm to remove background noise. This is then followed by extracting the foreground QR patterns from the background using a threshold. If the image is distorted, it is corrected using a Backpropagation (BP) neural network-based distortion correction method. If there is a perspective distortion, a perspective inverse transformation is applied.

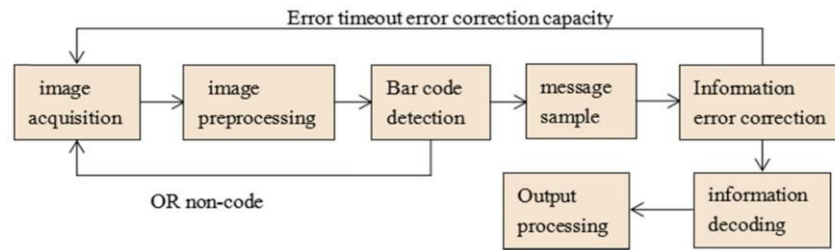


Figure 2.3.2 QR Code Recognition Process in [9]

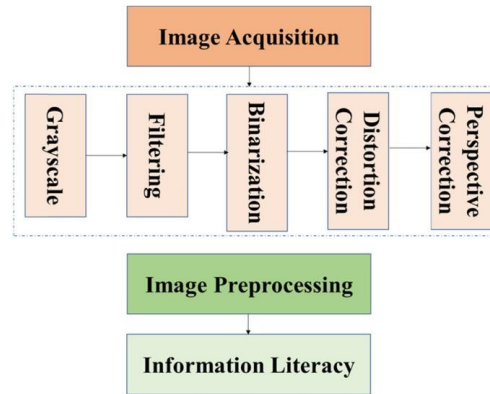


Figure 2.3.3 Image Preprocessing Process in [9]

2.4 Review of Technologies and Approaches in Indoor Navigation

Unlike traditional floor plan-based navigation solutions, [10] implemented a system that generates building architecture using Computer-Aided Design (CAD) models with a semantic mapping editor to manually update the map. The optimal path is computed with an A* algorithm, with an RGB-D camera capturing real-time depth data and images of the surroundings, a smart cane delivering haptic feedback, and a Time-Stamped Map (TSM) tracking obstacles to update navigation instructions. For user localization, [10] used the Singular Value Decomposition (SVD) algorithm to align the Area Description File (ADF) coordinate system with the semantic map, enabling accurate and consistent real-time localization within the environment.

Additionally, [10] used the TSM algorithm to create and update occupancy maps each frame, tracking obstacles dynamically as shown in Figure 2.4.1. However, the TSM system may face safety challenges in highly cluttered environments. To address this, [10] proposed the

usage of the Kalman Filter (KF) to estimate obstacle motion. KF will process dynamic obstacle data from TSM to estimate obstacle motion, allowing safer navigation in dynamic environments.

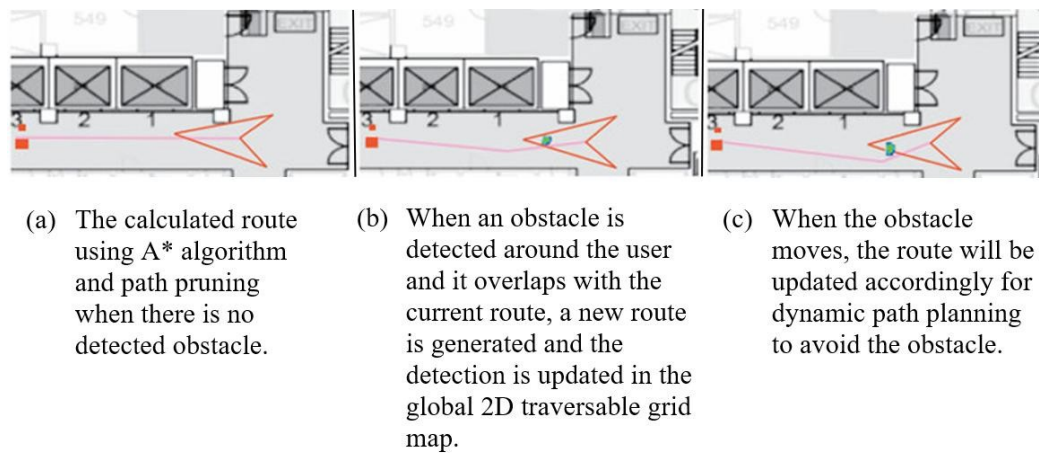


Figure 2.4.1: Navigation with Real-Time Obstacle Avoidance [10]

Additionally, a speech-to-text voice recognition module is integrated into a human-in-the-loop Human-Machine Interface (HMI), alongside cognitive waypoint path planning and smart cane support. This allows the system to provide general route guidance to the user, as illustrated in Figure 2.4.2.

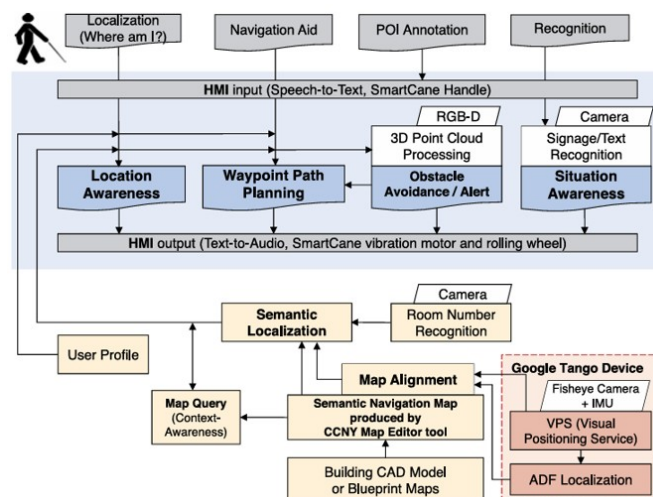


Figure 2.4.2: Navigation System Functional Block Diagram [10]

According to [11], a system was implemented that recognises specific scenes, providing situational awareness and offering navigation instructions as shown in Figure 2.4.3. For user localisation, [11] utilised the Speeded-Up Robust Features (SURF) algorithm, a vocabulary tree, a digital compass sensor, and a smartphone's built-in step-count sensor.

Based on [11], the authors employed the SURF algorithm, paired with a vocabulary tree, for efficient image recognition. This approach detects and matches visual features from incoming images with a pre-indexed database to identify the user's location. With this, the user's real-time location can be determined, improving navigation. The limitations of the SURF algorithm and vocabulary tree are the need for high-quality training data to achieve fast and reliable real-time image recognition.

Furthermore, [11] proposed the use of smartphone-based sensors to enhance user real-time localisation as an alternative. A compass sensor to provide directional data and a step-count sensor to track the user's real-time movement. However, the compass sensor may be prone to magnetic interference from surrounding objects and the step-count sensor may accumulate errors over long distances. To address this, [11] proposed integrating these sensors with the SURF algorithm and vocabulary tree to detect the user's current orientation and update routes in real time.

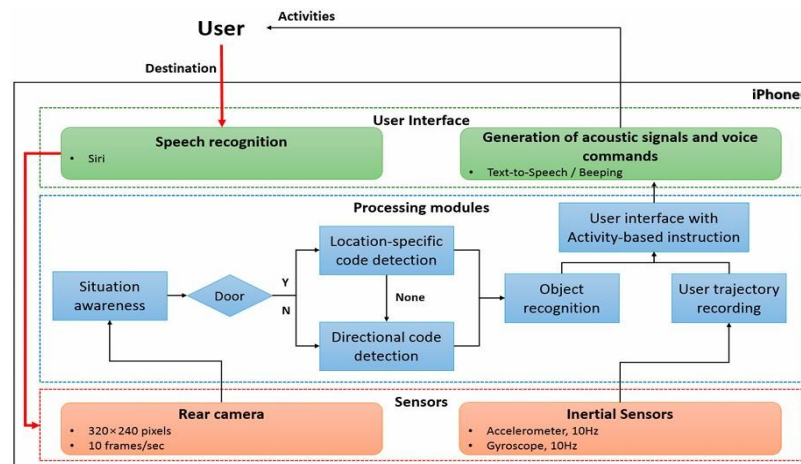


Figure 2.4.3: Architecture of The Navigation System [11]

2.5 Comparison of Methods

Done by: Every group member

2.5.1 Marker vs Marker-Less Approach

Table 2.5.1: Comparison Between Marker-Based and Markerless Approach

Feature/Aspect	Marker-Based Approach	Markerless Approach
Strength	Quick and efficient detection and localisation	No pre-installation of artificial markers
	More robust under varying light conditions	Adaptable to unfamiliar or dynamic environments
	Higher detection accuracy	No negative aesthetic impact on the environment
	Simpler detection process in controlled settings	High accuracy in position estimation
Weaknesses	Negative aesthetic impact due to physical marker placement	Require initial setup of feature/image databases or 3D models
	Effectiveness is limited by the marker installation	Sensitive to environmental changes
	Lighting conditions may affect QR code detection	High computational power and memory requirements
		High training data demand for AI-based methods
Shared Weakness	Lighting conditions, object occlusion, and dynamic scene elements may affect users' comfort due to the need to hold devices.	

2.5.2 Comparison of Colour Processing for QR Code Techniques

Table 2.5.2 : Comparison of Colour Processing Techniques

Techniques	Advantages	Disadvantages
Forest Fire Recognition Based on Feature Extraction from Multi-View Images [6]		
HSV Colour Space + Gaussian Mixture Model (GMM)	Provides an intuitive and effective representation of colours	Requires large dataset for training purpose

	Reduces computational complexity by projecting the 3D HSV distribution onto three 2D planes	Computational cost of training and modelling GMMs can be high.
Color Image Processing: Methods and Application (Chapter 5) [7]		
K-means clustering	Unsupervised clustering based on colour similarity	Performance depends on the initial clusters' centers
	Simple and easy to implement	May require post-processing to handle noise and over-segmentation
Region growing	Groups neighbouring pixels based on colour similarity and spatial proximity	Sensitive to the initial seed placement and parameter selection
	Seeded and unseeded variants allow flexible application.	Result can be wrong if seed or homogeneity criteria are poorly chosen.

2.5.3 Comparison of QR Code Detection Techniques

Table 2.5.3: Comparison of QR Code Detection Techniques

Techniques	Advantages	Disadvantages
Localization and Navigation Using QR Code in the Indoor Environment [8]		
QR Code Grid Map System	Enables accurate position localization in indoor environments	Static and environment dependent as system fails if QR codes are distorted.
Camera-based Image Capture	Uses a simple industrial camera with upward orientation, suitable for fixed ceiling-mounted QR codes.	Detection accuracy can be affected by insufficient lighting, motion blur, or if the robot moves too fast.

QR Code Detection and Decoding (OpenCV and ZBar)	Fast and lightweight decoding pipeline	Performance may degrade under poor image quality or when QR codes are distorted.
Research on QR image code recognition system based on artificial intelligence algorithm [9]		
Improved Adaptive Median Filter	Improved accuracy in detecting noise points.	May increase the number of times the filter window has to expand.
BP Neural Network for Distortion Correction	Effectively corrects surface distortion.	Requires a large dataset of training samples.
OSTU algorithm for binarization	Simplifies the QR code into binary (black/white) for decoding	Not effective on images with complex backgrounds or severe noise.
Perspective Projection Inverse Transformation	Allows QR code to be properly interpreted even after severe geometric warping.	Not effective alone as it requires good distortion correction beforehand.

2.5.4 Comparison of Navigation Techniques

Table 2.5.4: Comparison of Navigation Approaches

Technique Used	Advantages	Disadvantages
Vision-Based Mobile Indoor Assistive Navigation Aid for Blind People [10]		
CAD Models + Semantic Map Editor	Supports path planning from building data.	Needs accurate CAD models and manual updates.
A* Algorithm	Finds optimal path efficiently on grid maps.	High computation time in dynamic environments.
SVD for ADF Alignment	Improves localization accuracy.	Sensitive to localization drift if the environment changes.

Time-Stamped Map TSM	Detects obstacles in real time.	Struggle in cluttered environments.
Kalman Filter	Predicts obstacle motion.	May not match real-world behavior.
Cognitive Waypoint Path Planning	Provide step-by-step guidance for navigation.	Complex in dynamic layouts.
A Vision-Based Wayfinding System for Visually Impaired People Using Situation Awareness and Activity-Based Instructions [11]		
SURF algorithm	Fast, reliable for real-time image recognition.	Performance drops in cluttered areas.
Vocabulary Tree	Recognizes many scenes quickly.	Time-consuming to build and requires good training data.
Digital compass	Resolve direction ambiguity.	Prone to magnetic interference.
Built-in Step-Count Sensor	No external hardware and low cost.	May accumulate errors over long distances (drift issue).

CHAPTER 3

Proposed Method/Approach

3.1 Design Specification

3.1.1 Methodologies and General Work Procedures

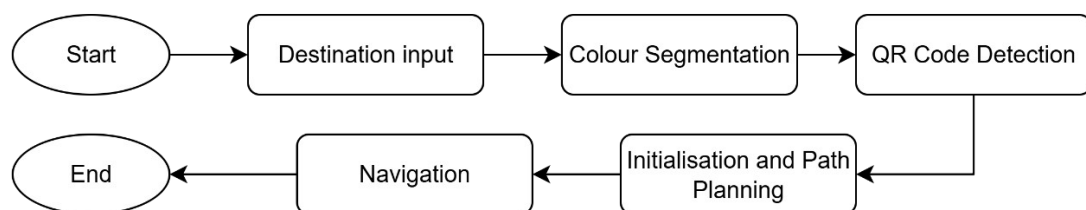


Figure 3.1.1.1: System Flowchart for General Work Procedures

CHAPTER 3

The general approach of this project begins with the auditory input of the destination. This auditory input is necessary for the subsequent optimal route calculation. Once the destination has been confirmed, the system concurrently performs colour segmentation and QR code detection. The use of colour segmentation simplifies environmental analysis by filtering specific colours — red, green, or blue — from objects. Simultaneously, QR code detection identifies encoded location markers. When a QR code is detected, the encoded information is fed into the initialisation and path planning stage, where the optimal path is computed. Finally, the navigation module guides the user along the planned path, completing the process.

3.1.2 Tools to Use

Hardware Used

Table 3.1.2.1: Hardware Specifications

Description	Specifications
Operating System (OS)	Windows 11 Enterprise
Central Processing Unit (CPU)	Intel Core i7- 14700K
Graphics Processing Unit (GPU)	Gigabyte GeForce RTX 4070 Super 12GB GDDR6
Random-Access Memory (RAM)	32GB DDR5 RAM
Storage	512GB SSD

Software

The following would be the required tools for the proposed system:

1. Microsoft Visual Studio 2022 Enterprise Edition
2. OpenCV (version 4.11.0)
3. Visual Studio Code
4. Google Colab

3.1.3 User Requirements

- The system should be able to detect red, green, or blue objects.
- The system should automatically detect and recognise QR codes even with object occlusions without any user intervention, ensuring a hands-free experience.

CHAPTER 3

- The system should maintain consistency across varying lighting conditions, without requiring user adjustments.
- Users should be able to input their desired destination via audio.
- The system should define a route to the destination when the QR code is scanned.
- The system should guide the user to the desired destination via audio and haptic feedback.
- The system should detect deviation from the defined route and reroute the user back to the optimal route.

3.1.4 System Performance Definition

The system performance definition for this system revolves around achieving targeted improvements in QR detection accuracy and timing, route optimisation, and overall reliability to ensure optimal functionality and user satisfaction.

The system aims to achieve a minimum of 90% accuracy for colour segmentation under various lighting conditions – low light or excessively bright lighting, effectively reducing verification complexity. It is pertinent for the system to process and identify QR codes within 100 milliseconds to support real-time detection and seamless transition from detection to routing. It is also important to keep the latency between QR code detection and routing within 300 milliseconds.

The system should prioritise the nearest relevant QR code when multiple are present, avoiding ambiguity. Audio input recognition should maintain $\geq 92\%$ accuracy. The audio and haptic feedback should occur within 150 milliseconds of navigation or route adjustments.

3.1.5 Verification Plan

1. Orientation and Position Variability: The system will be tested with cameras positioned at various angles and distances. This will assess the system's ability to detect coloured objects and filter out QR codes.

2. **Lighting Conditions:** The QR codes will be placed under various lighting conditions to mimic real life to assess the system's ability to recognise QR codes. This includes conditions such as low light and overexposure.
3. **Colour Variations:** The system should be able to segment colours of varying ages as realistically; printed colours would fade over time. This ensures that the QR codes do not have to be reprinted over a longer period.
4. **Occlusion:** Objects will be placed on the QR codes to partially cover them, mimicking a cluttered environment. This is to ensure that the system can still identify QR Codes accurately in an unpredictable indoor environment.
5. **Multiple QR Codes in a Single Frame:** This test will involve multiple QR codes placed within the same field of view. The system should choose the QR code nearest to the user.

3.2 System Design Diagram/Equation

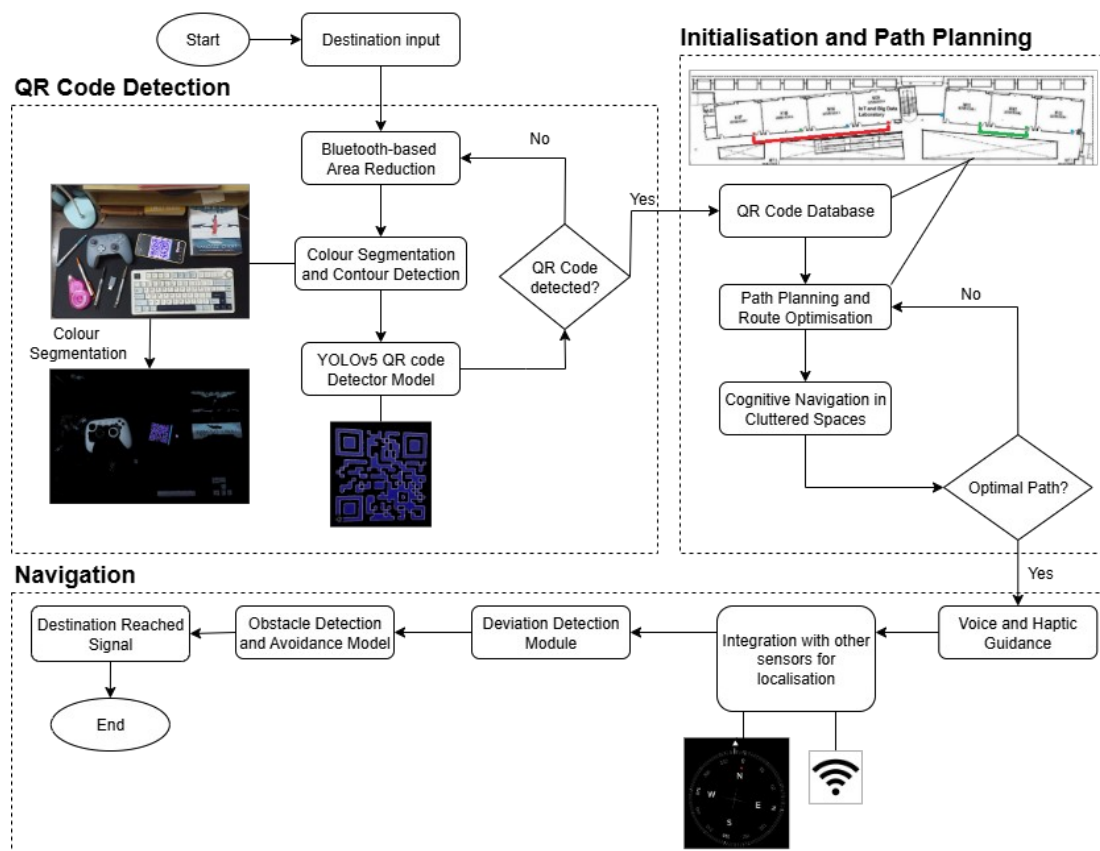


Figure 3.2.1: System Overview Block Diagram

3.2.1 Destination Input

The user's desired destination is captured auditorily with the smartphone's built-in microphone and then converted to text using a speech recognition engine. The existence of the desired destination is verified with a location ID stored in a database. The location ID is sent to the initialisation and path planning module for route planning.

3.2.2 QR Code Detection

This stage focuses on identifying and processing coloured QR codes. If a QR code is detected, the system will provide a step-by-step guidance to navigate the user to the most optimal position to read the QR code.

3.2.2.1 Bluetooth-based Area Reduction

This module aims to reduce the number of QR codes to verify by gauging the Bluetooth signal strength from nearby beacons and defining a general area to focus on; each coloured QR code location is installed with a dedicated Bluetooth beacon.

3.2.2.2 Colour Segmentation and Contour Detection

For enhanced robustness towards lighting changes and to improve colour filtering effectiveness, the frames are converted from RGB to HSV colour space. A predefined HSV range is applied according to the chosen colour – Red, Green, or Blue – to generate a binary mask that highlights only the regions matching the target colour. A contour detection technique will be applied to the segmented regions to detect boundaries that match QR code geometry, typically square or near-square shapes. These result in a cleaner, more focused output that enhances detection speed and accuracy.

3.2.2.3 Yolov5 QR Code Detector Model

A fine-tuned YOLOv5 model is utilised in the system. The model is trained using a labelled coloured QR code dataset in varied lighting conditions. If there is no QR code detected within the segmented region, the system returns to the Bluetooth-based area reduction module

CHAPTER 3

to guide the user to the nearest Bluetooth beacon. The detected QR code would be passed onto the initialisation and path planning module.

3.2.3 Initialisation and Path Planning

3.2.3.1 QR Code Database

The system retrieves QR code coordinates from the database. This process identified the user's location in the building as each QR code is mapped to a specific location within the building. The database stores each QR code's physical location and the shortest, logical path to other QR codes.

3.2.3.2 Path Planning and Route Optimisation

A* algorithm is used to compute the shortest route from one QR code to another by combining the actual travel distance from the starting point with an admissible heuristic estimate of the remaining path. This enables rapid identification of the optimal path. Precomputing distances minimises processing load during navigation, enhancing system efficiency.

3.2.3.3 Cognitive Navigation Cluttered Spaces

This module evaluates whether the current optimal path is cluttered or blocked to ensure the navigation remains safe for the user. If any potential obstruction is detected, the system returns to the path planning and route optimisation stage to recompute an alternative optimal path.

3.2.4 Navigation

3.2.4.1 Voice and Haptic Guidance

Navigation instructions are given through voice and haptic feedback. Voice guidance is delivered via the smartphone's speaker at controlled intervals to ensure clarity and prevent information overload. Haptic feedback is given via the vibration motor of the smartphone; strong vibration signals near objects, while gentle vibrations indicate distance to the next

CHAPTER 3

waypoint. As the user nears a turn, vibration intensity increases gradually without becoming overwhelming. The combination of audio and haptic delivers clear, intuitive guidance and enhances the overall navigation experience.

3.2.4.2 Integration with other sensors for localisation

To improve users' localisation, the system integrates both compass and Wi-Fi signal strength. The digital compass can determine the user's facing direction, enhancing orientation precision during navigation. For Wi-Fi positioning, the system measures the Received Signal Strength Indicator (RSSI) values from nearby access points to estimate the user's location within the building.

3.2.4.3 Deviation Detection Module

To ensure that the user remains within an allowable range of the planned route, the system would redirect the user back onto the optimal route. This is also achieved with audio and haptic feedback.

3.2.4.4 Obstacle Detection and Avoidance Model

During navigation, a real-time obstacle detection along the user's path is also implemented. Upon the detection of an obstacle, the system triggers haptic feedback to alert the user and provides voice guidance to instruct the user to safely avoid the obstruction.

3.2.4.5 Destination Reached Signal

Once the user reaches the target destination, the system confirms arrival with a voice notification and a consistent haptic vibration pattern to inform the user. The system compares the user's current location with the destination coordinates to determine arrival.

3.3 Implementation Issues and Challenges

A technical challenge encountered during the development of the system is maintaining the high accuracy of QR code detection. The detection accuracy may vary due to several reasons, such as poor lighting conditions, glare, or shadow.

Furthermore, a large dataset of QR codes needs to be collected to fine-tune YOLOv5; a QR code dataset must contain images that are captured under various lighting conditions, angles, and distances to reflect real-world conditions. Hence, without sufficient variety and volume of fine-tuning data, the model's performance may be subpar when deployed in unseen or challenging environments. For instance, without sufficient training, the YOLOv5 model may still struggle with blurry or partially obscured QR codes.

Additionally, although the A* algorithm allows the computation of the shortest path from the scanned location to the destination, it does not take obstacles - students walking in hallways, animals, and moving objects in real time - into consideration. These obstacles may appear after the path is calculated, causing deviation and requiring the algorithm to execute again, increasing computational load and consequently slowing down navigation.

3.4 Timeline and Milestone for 14 Weeks

Figure 3.4.1 illustrates the tasks in a simplified format through a milestone chart, while Figure 3.4.2 presents the project timeline over 14 weeks using a Gantt chart. Both charts outline the details of executing this project. By utilizing these visual schedules, we can track progress and identify task priorities or non-critical tasks, allowing us to manage time effectively.

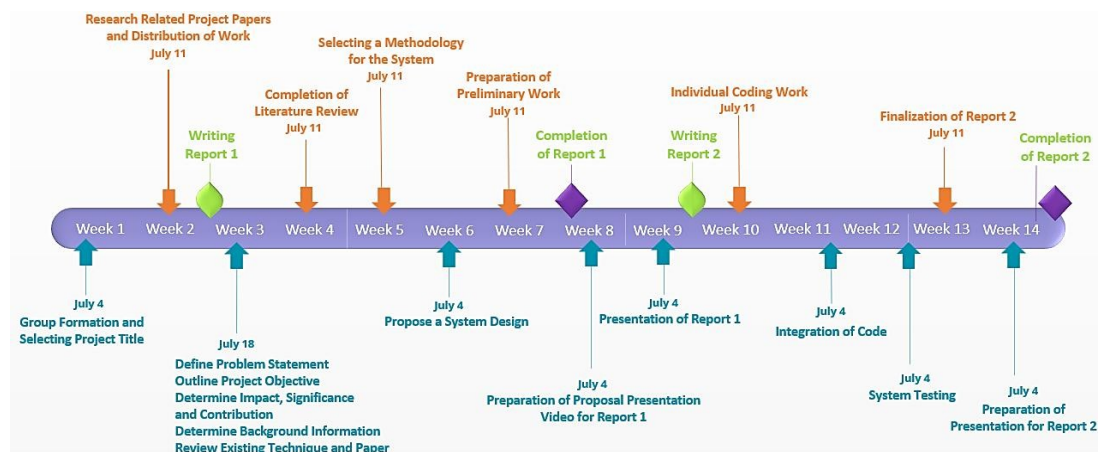


Figure 3.4.1: Milestones Chart of The Project

CHAPTER 3

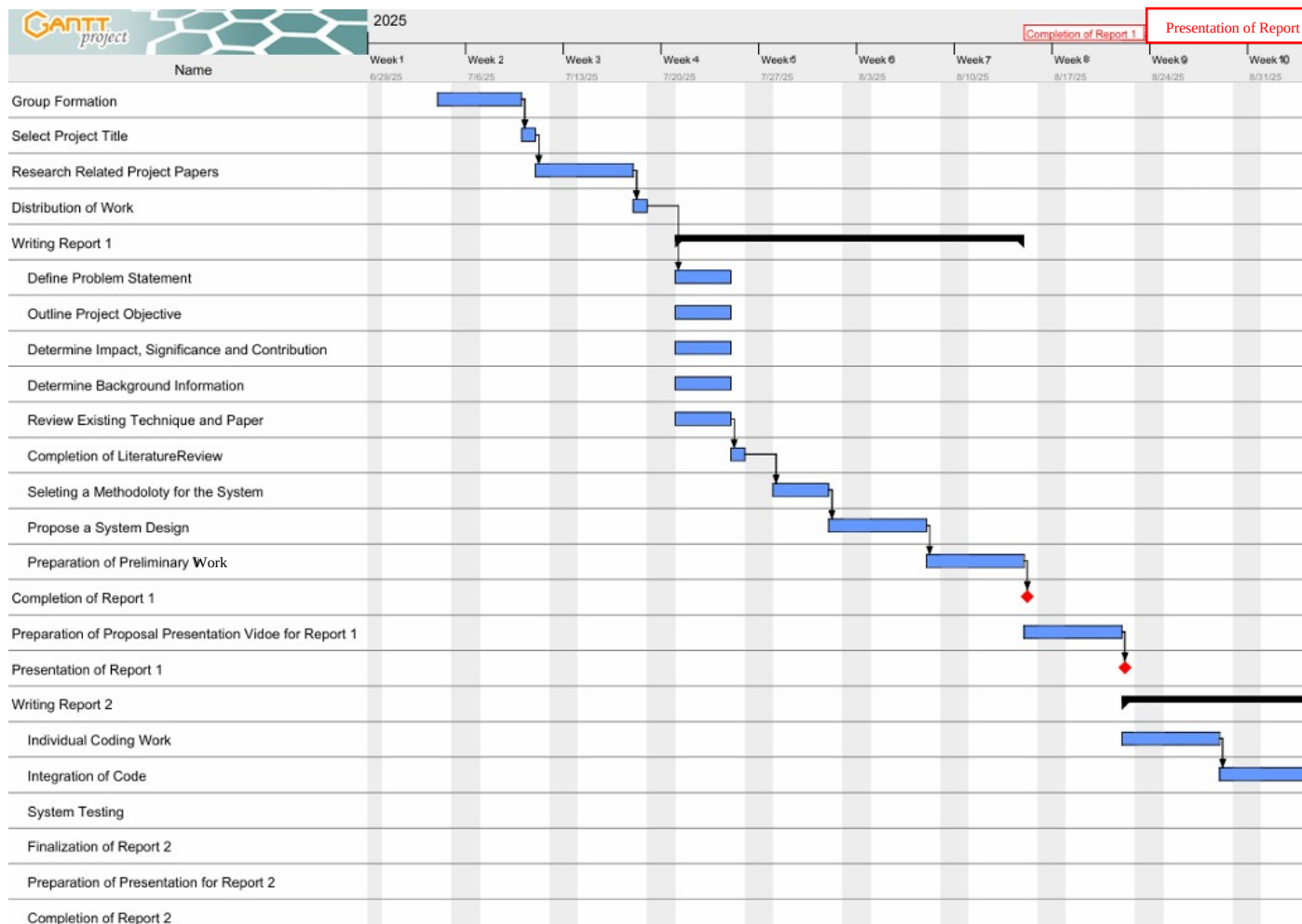


Figure 3.4.2: Gantt Chart for Project Timeline

Bachelor of Computer Science (Honours)
Faculty of Information and Communication Technology (Kampar Campus), UTAR

CHAPTER 4

Preliminary Work

4.1 Preliminary Work 1

Since the target users are VI individuals, less emphasis is placed on creating an attractive UI, but on auditory output. The text shown is meant for demonstration and debugging purposes. The user first announces the desired destination and then decides which QR code colour – red, green, or blue – for the system to detect (Figure B.1.1). Once chosen, the system would confirm the user's selection via audio (Figure B.1.2) and begin scanning for the QR code to determine a starting point. If a QR code is caught within the user's vicinity, the system will draw a bounding box around the QR code and guide the user towards the QR code, as shown in Figure B.1.3 and Figure B.1.4. Once the user is within range or the QR code is readable, the system will announce the content of the QR code and begin to guide the user towards the desired destination (Figure B.1.5).

If the colour of the detected QR code is the wrong colour, the user would be notified as well, as shown in Figure B.1.6.

Currently, the user is required to be positioned strictly in the centre, which is inconvenient to VI individuals. So, a possible improvement is to define an acceptable area in which a QR code can be scanned and read.

4.2 Preliminary Work 2

As an initial step, the chosen colour of the QR code was extracted from its background using OpenCV in Python. The function is responsible for performing this step by applying HSV thresholds and masking, as shown in Appendix B. According to Table 4.2.1, the system performance evaluation was carried out under varying QR code areas, QR code patterns,

CHAPTER 4

detection ranges, and two lighting conditions, namely low light (sunlight) and bright light (sunlight combined with room lighting).

Table 4.2.1: QR Code Detection Result Under Various Conditions

QR code pattern	QR code area (cm ²)	lighting conditions	Result of QR Code Detection under different Detection Range (cm)		
			50	100	150
Red module + white background	81	Low light	NO	NO	NO
	81	Bright	YES	NO	NO
	324	Low light	YES	NO	NO
	324	Bright	YES	YES	NO
White module + red background	81	Low light	NO	NO	NO
	81	Bright	YES	NO	NO
	324	Low light	YES	NO	NO
	324	Bright	YES	YES	NO
Red module + black background	81	Low light	NO	NO	NO
	81	Bright	YES	NO	NO
	324	Low light	YES	NO	NO
	324	Bright	YES	YES	NO
Black module + red background	81	Low light	NO	NO	NO
	81	Bright	YES	NO	NO
	324	Low light	YES	NO	NO
	324	Bright	YES	YES	NO

As shown in Table 4.2.1, the system achieved a successful detection for QR codes with an area of 81 cm² at the detection distance of 50 cm under both lighting conditions, regardless of the QR code pattern. A larger QR code area of 324 cm² is generally more detectable at large distances than the smaller QR code. Moreover, bright lighting conditions consistently result in more successful detections across different QR code patterns and areas compared to low light condition. As a result, the system can perform reliable QR code detection at distances of less than 100 cm when larger QR codes are used under bright lighting conditions.

4.3 Preliminary Work 3

The ZBar library was chosen for decoding the detected QR codes instead of OpenCV due to its higher success rate (83.30%) compared to OpenCV (61.69%), as shown in Appendix B, Figure B.3.1. Figure B.3.2 in Appendix B presents examples of QR code images that were successfully decoded. For testing, a QR code dataset from Kaggle which consists of 509 QR code images in real-time settings was used [12].

Before decoding the QR code images, preprocessing was applied to improve decoding accuracy using OpenCV functions as shown in Figure 4.3.1. First, each QR code image is converted to grayscale to reduce data size and computational requirements. Then, perspective correction is performed on the grayscale image to crop and straighten tilted or skewed QR codes. The image is then warped into a corrected, front-facing view. Next, the perspective-corrected image is upscaled, making the squares in the QR code more visible for better decoding. Finally, the upscaled image is binarized, converting it into a pure black-and-white format to make the QR code's pattern cleaner and easier for the decoder to interpret.

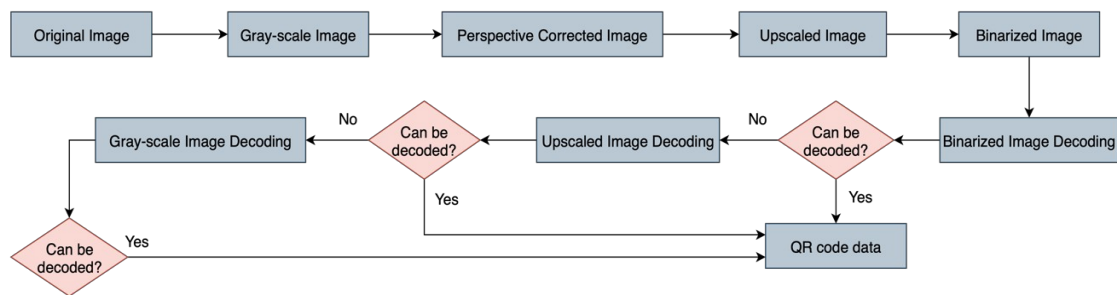


Figure 4.3.1: QR Code Image Preprocessing Steps Before Decoding

However, binarization may cause some dark squares to be turned white, leading to decoding failures, as shown in, Appendix B, Figure B.3.3. To address this, the upscaled image is used for decoding if the binarized image fails. Additionally, perspective correction may fail or overcrop the QR code image, also resulting in decoding failures, as illustrated in Appendix B, Figure B.3.4. In such cases, the grayscale image is used for decoding.

As shown in Appendix B, Figure B.3.5, 384 out of 424 QR code images were successfully decoded using binarized images, 23 were decoded using upscaled images, and 17 were decoded using grayscale images. A total of 85 QR code images could not be decoded due to light reflections, shadows, or distortion, as shown in Appendix B, Figure B.3.6, Figure B.3.7, and Figure B.3.8. In future work, the decoding results will be further enhanced by applying new processing methods or by improving the current ones.

4.4 Preliminary Work 4

A coordinate-based data structure is chosen to create a map of the University of Tunku Abdul Rahman's (UTAR) Faculty of Information and Communication Technology (FICT) ground floor. This approach is flexible towards irregular room shapes in UTAR, facilitating precise navigation in a real-world environment.

The FICT ground floor building comprises two main rooms: the lecturer offices and the classrooms. The classroom section in the map structure was created and an A* algorithm was implemented to find the optimised routes. By referring to Appendix B, Figure B.4.1, the classroom layout is created. The classrooms were made in sequence with their adjacent and opposite objects, as shown in Appendix B, Figure B.4.2.

Based on Appendix B Figure B.4.3, the A* algorithm for pathfinding is tested using N001 as the starting point to all available destinations. The output displays the total distance and the path details. We noticed that the created map aligns accurately with the ground floor plan of UTAR FICT, which is shown in Appendix B Figure B.4.4. In summary, the coordinate-based map structure integrates seamlessly with the A* algorithm. For future work, the remaining office layout will be developed and incorporated into the existing map for integration with the A* algorithm.

CHAPTER 5

Conclusion

5.1 Summarisation of Finding

The main objective of this project was to develop an efficient, low-cost indoor navigation system for VI individuals using colour-coded QR codes. The proposed system aimed to deliver accurate real-time navigation, robustness in varying lighting conditions, and hands-free operation through voice-guided assistance. Compared to existing indoor navigation solutions, this approach focuses on enhancing accessibility while maintaining affordability and ease of deployment.

Our findings show that integrating colour segmentation, YOLOv5-based QR code detection, and the A* pathfinding algorithm enables reliable localisation and route planning in indoor environments. The system can accurately detect QR codes under diverse conditions, decode location data, and provide step-by-step audio guidance to the user. This reduces dependence on external assistance and enhances user autonomy.

Testing revealed that while the system performs well in most scenarios, challenges remain with QR code detection under poor lighting, glare, or partial occlusion, as well as with handling dynamic obstacles that require real-time route recalculation. These limitations highlight the need for further optimisation of detection algorithms, expansion of the training dataset, and integration of more adaptive obstacle-avoidance techniques.

Overall, the proposed system demonstrates significant potential to improve safety, mobility, and independence for VI individuals in indoor environments, paving the way for more inclusive and accessible navigation solutions.

5.2 Novelties of Work

The main novelty of this system is the use of colour-coded QR codes. With the use of colours, the system can perform targeted scanning, allowing the camera to focus only on regions or objects matching the predefined colour profiles. This can minimise false detections in cluttered areas.

Another element of novelty is the use of a deep learning model to detect and identify QR codes. Unlike traditional methods that rely on rigid pattern matching and handcrafted features, deep learning models offer higher accuracy and robustness under challenging conditions such as partial occlusions, variable lighting, image distortion, and background clutter, all without manual tuning or environmental-specific adjustments.

Additionally, to simplify QR code selection, a Bluetooth-based area reduction strategy is implemented. QR codes located in areas where Bluetooth signal strength is the strongest relative to the users are prioritised for detection. Alongside Bluetooth, data from other sensors such as IMU and Wi-Fi signals are used to improve localisation by helping to establish a general area or movement trajectory before or between QR codes.

In terms of path planning and navigation, computing the shortest path to the destination is not sufficient, so a machine learning module is integrated to learn and adapt to the user's navigation behaviour over time. In cluttered or complex spaces, such adaptive navigation helps reduce confusion and improve overall user experience.

5.3 Concluding with Supportive Remark

Done by: Kuek Levin

This project has developed a QR code-based indoor navigation system that effectively addresses the challenges faced by the VI. By integrating techniques such as a colour-based object detection algorithm, YOLOv5 model, and A* algorithm, a high degree of accuracy and reliability is achieved. This is demonstrated by its ability to consistently detect QR codes under a variety of lighting conditions and provide real-time instructions, which guide users to reach their desired destinations independently and safely.

The use of colour segmentation allows the system to filter and keep the object that meets the expected colour and shape of the QR code, which improves the detection accuracy. Additionally, a YOLO model contributes to a more efficient QR code detection, enhancing the robustness and the real-time processing capability of the system. The interaction between these components guarantees that the entire navigation process remains smooth and reliable. Finally, the system computes the most optimal route with the A* algorithm to guide the users to their destination.

In conclusion, this project demonstrates a strong potential for real-world application with the combination of technical feasibility and social impact. This contribution is valuable to the field of computer vision while improving accessibility and independence for the visually impaired. In the long run, this work gives the vulnerable populations more equality and allows smoother integration into society.

BIBLIOGRAPHY/REFERENCES

- [1] J. M. Feghali, C. Feng, A. Majumdar, and W. Y. Ochieng, “Comprehensive Review: High-Performance Positioning Systems for Navigation and Wayfinding for Visually Impaired People,” *Sensors*, vol. 24, no. 21, pp. 7020–7020, Oct. 2024, doi: <https://doi.org/10.3390/s24217020>. (accessed Jul. 17, 2025).
- [2] H. A. Karimi, *Indoor Wayfinding and Navigation*. CRC Press, 2015. Accessed: Dec. 08, 2025. [Online]. Available: https://api.pageplace.de/preview/DT0400.9781482230857_A24326319/preview-9781482230857_A24326319.pdf (accessed Jul. 17, 2025).
- [3] A. Morar *et al.*, “A Comprehensive Survey of Indoor Localization Methods Based on Computer Vision,” *Sensors*, vol. 20, no. 9, p. 2641, May 2020, doi: <https://doi.org/10.3390/s20092641>. (accessed Jul. 17, 2025).
- [4] D. Khan, Z. Cheng, H. Uchiyama, S. Ali, M. Asshad, and K. Kiyokawa, “Recent advances in vision-based indoor navigation: A systematic literature review,” *Computers & Graphics*, vol. 104, no. 24, pp. 24–45, May 2022, doi: <https://doi.org/10.1016/j.cag.2022.03.005>.
- [5] F. Zafari, A. Gkelias, and K. K. Leung, “A Survey of Indoor Localization Systems and Technologies,” *IEEE Communications Surveys Tutorials*, vol. 21, no. 3, pp. 2568–2599, 2019, doi: <https://doi.org/10.1109/COMST.2019.2911558>. (accessed Jul. 17, 2025).
- [6] D. Wu, C. Zhang, L. Ji, R. Ran, H. Wu, and Y. Xu, “Forest Fire Recognition Based on Feature Extraction from Multi-View Images,” *Traitement Du Signal*, vol. 38, no. 3, pp. 775–783, Jun. 2021, doi: <https://doi.org/10.18280/ts.380324>. (accessed Jul. 17, 2025).
- [7] R. Lukac and K. N. Plataniotis, Eds., *Color Image Processing*. CRC Press, 2018. doi: <https://doi.org/10.1201/9781315221526>. (accessed Jul. 17, 2025).
- [8] H. Zhang, C. Zhang, W. Yang, and C.-Y. Chen, “Localization and navigation using QR code for mobile robot in indoor environment,” *2015 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, Dec. 2015, doi: <https://doi.org/10.1109/robio.2015.7419715>.

REFERENCES

- [9] L. Huo, J. Zhu, P. K. Singh, and P. A. Pavlovich, "Research on QR image code recognition system based on artificial intelligence algorithm," *Journal of Intelligent Systems*, vol. 30, no. 1, pp. 855–867, 2021, doi: <https://doi.org/10.1515/jisys-2020-0143>.
- [10] B. Li *et al.*, "Vision-Based Mobile Indoor Assistive Navigation Aid for Blind People," *IEEE Transactions on Mobile Computing*, vol. 18, no. 3, pp. 702–714, Mar. 2019, doi: <https://doi.org/10.1109/tmc.2018.2842751>. (accessed Jul. 17, 2025).
- [11] E. Ko and E. Y. Kim, "A Vision-Based Wayfinding System for Visually Impaired People Using Situation Awareness and Activity-Based Instructions," *Sensors*, vol. 17, no. 8, p. 1882, Aug. 2017, doi: <https://doi.org/10.3390/s17081882>. (accessed Jul. 17, 2025).
- [12] vincent-vdb, "QR-detection-yolo," *Kaggle.com*, 2023.
<https://www.kaggle.com/datasets/vincentv/qr-detection-yolo> (accessed Aug. 15, 2025).

APPENDIX A

Code Sample

A.1 Preliminary Work 1 Codes

A.1.1 Important Libraries

```
#include "NavigationController.h"
#include <iostream>
#include <algorithm>
#include <chrono>
#include <thread>
```

A.1.2 Setting QR Code Colours

```
void NavigationController::setQRColor(int color) {
    qrDetector.setTargetColor(color);
    std::string colors[] = { "RED", "GREEN", "BLUE" };
    if (color >= 0 && color <= 2) {
        audioManager.speakInstruction("QR code color set to " + colors[color]);
    }
}
```

A.1.3 Guiding Users Towards QR Code

```
void NavigationController::providePreciseDirections(const cv::Point2f& qrCenter, const
cv::Point2f& screenCenter) {
    static auto lastDirectionTime = std::chrono::steady_clock::now();
    static std::string lastDirection = "";

    auto now = std::chrono::steady_clock::now();
    auto timeSinceLastDirection = std::chrono::duration_cast<std::chrono::milliseconds>(now -
lastDirectionTime);

    // Throttle direction updates to prevent audio spam (minimum 1.5 seconds between directions)
    if (timeSinceLastDirection.count() < 1500) {
        return;
    }

    cv::Point2f direction = qrCenter - screenCenter;

    // Calculate angle and distance
    float angle = atan2(direction.y, direction.x) * 180.0f / CV_PI;
```

```

float distance = sqrt(direction.x * direction.x + direction.y * direction.y);

std::string newDirection;

// Provide specific directional guidance based on distance and angle
if (distance > 100) {
    if (angle > -45 && angle <= 45) {
        newDirection = "Turn right to find the QR code";
    }
    else if (angle > 45 && angle <= 135) {
        newDirection = "Look down and right for the QR code";
    }
    else if (angle > 135 || angle <= -135) {
        newDirection = "Turn left to find the QR code";
    }
    else {
        newDirection = "Look up and left for the QR code";
    }
}
else if (distance > 50) {
    if (abs(direction.x) > abs(direction.y)) {
        if (direction.x > 0) {
            newDirection = "Turn slightly right";
        }
        else {
            newDirection = "Turn slightly left";
        }
    }
    else {
        if (direction.y > 0) {
            newDirection = "Look down slightly";
        }
        else {
            newDirection = "Look up slightly";
        }
    }
}
else {
    newDirection = "QR code is centered. Move closer to read it";
}

// Only speak if the direction has changed or enough time has passed
if (newDirection != lastDirection) {
    AudioManager.speak(newDirection, true); // Use async speech to prevent blocking
    lastDirection = newDirection;
    lastDirectionTime = now;
}
}

```

A.1.4 Auditory Rotation Guidance

```

bool NavigationController::performRotationGuidance(const std::string& direction) {
    currentState = NavigationState::ROTATING;
    audioManager.speakDirection("Turn " + direction + " slowly. I will tell you when to stop.");

    cv::Mat frame;
    double bestSimilarity = 0.0;
    const double SIMILARITY_THRESHOLD = 0.75;
    const double PROGRESS_THRESHOLD = 0.1;
    double lastReportedSimilarity = 0.0;
    auto lastFeedbackTime = std::chrono::steady_clock::now();

    cv::Mat referenceImage = cameraManager.getReferenceImage();
    if (referenceImage.empty()) {
        audioManager.speakInstruction("Error: No reference image available.");
        currentState = NavigationState::SCANNING;
        return false;
    }

    while (currentState == NavigationState::ROTATING && isRunning) {
        if (!cameraManager.captureFrame(frame)) continue;

        double similarity = cameraManager.calculateSimilarity(frame, referenceImage);
        int percentMatch = static_cast<int>(similarity * 100);

        std::string status = "TURN " + direction + " | Match: " + std::to_string(percentMatch) + "%";

        auto now = std::chrono::steady_clock::now();
        auto timeSince = std::chrono::duration_cast<std::chrono::seconds>(now - lastFeedbackTime);

        // Provide progress feedback
        if (similarity - lastReportedSimilarity > PROGRESS_THRESHOLD && timeSince.count() >
2) {
            if (similarity > 0.5) {
                audioManager.speakDirection("Good progress. " + std::to_string(percentMatch) + "
percent aligned.");
                lastReportedSimilarity = similarity;
                lastFeedbackTime = now;
            }
        }

        // Check if alignment is achieved
        if (similarity > SIMILARITY_THRESHOLD) {
            audioManager.speakDirection("Perfect alignment! You may now walk forward. I'll continue
scanning for the next QR code.");
            audioManager.playSuccessSound();
        }
    }
}

```

```

        currentState = NavigationState::SCANNING;
        return true;
    }

    // Warn if moving in wrong direction
    if (similarity < 0.2 && bestSimilarity > 0.4 && timeSince.count() > 3) {
        audioManager.speakDirection("You're moving away from the target. Try rotating the other
direction.");
        lastFeedbackTime = now;
    }

    if (similarity > bestSimilarity) {
        bestSimilarity = similarity;
    }

    // Display frame with status
    cameraManager.drawStatusInfo(frame, status, qrDetector.getTargetColor());
    cameraManager.displayFrame(frame, status);

    // Handle ESC key to exit rotation mode
    char key = cv::waitKey(33);
    if (key == 27) { // ESC
        currentState = NavigationState::SCANNING;
        return false;
    }
}
currentState = NavigationState::SCANNING;
return false;
}

```

2.2 Preliminary Work 2 Codes

A.2.1 Important Libraries

```

import cv2 as cv
import numpy as np

```

A.2.2 QR Code Detection

```

"""
    Captures video from a webcam, detects blue, green, and red colors, and displays the original and
    color-isolated frames.
"""
# 0 for the default camera
def detect_target_colors(camera_index=0):

    # create object to access the camera
    cap = cv.VideoCapture(camera_index)

```

```

# check is the camera open
if not cap.isOpened():
    print("Error: Could not open video stream.")
    return

# continuously process video frames
while True:

    # read a frame from the camera (_ ignores the success boolean, frame gets the image)
    _, frame = cap.read()

    # check is frame is successfully captured, if no exit
    if frame is None:
        break

    # flip frame horizontally for mirror effect
    frame = cv.flip(frame, 1)
    # convert frame to hsv format
    hsv = cv.cvtColor(frame, cv.COLOR_BGR2HSV)
    # --- Red Color Detection ---
    # set HSV threshold for first red range (0-10 degrees in hue circle)
    lower_red1 = np.array([0, 50, 50])
    upper_red1 = np.array([10, 255, 255])
    # set HSV threshold for second red range (170-180 degrees in hue circle)
    lower_red2 = np.array([170, 50, 50])
    upper_red2 = np.array([180, 255, 255])

    # --- Blue Color Detection ---
    # set HSV threshold for blue color detection
    lower_blue = np.array([100, 50, 50])
    upper_blue = np.array([140, 255, 255])

    # --- Green Color Detection ---
    # set HSV threshold for green color detection
    lower_green = np.array([40, 50, 50])
    upper_green = np.array([80, 255, 255])

    # create and apply the blue binary mask to original frame to isolate blue regions (blue = white,
other = black)
    mask_blue = cv.inRange(hsv, lower_blue, upper_blue)
    res_blue = cv.bitwise_and(frame, frame, mask=mask_blue)

    # create and apply the blue binary mask to original frame to isolate green regions
    mask_green = cv.inRange(hsv, lower_green, upper_green)
    res_green = cv.bitwise_and(frame, frame, mask=mask_green)

    # create binary mask for first and secon red range
    mask_red1 = cv.inRange(hsv, lower_red1, upper_red1)
    mask_red2 = cv.inRange(hsv, lower_red2, upper_red2)

    # use bitwise OR operation to combine mask
    mask_red = cv.add(mask_red1, mask_red2)
    # Apply the combined red mask to original frame to isolate red regions
    res_red = cv.bitwise_and(frame, frame, mask=mask_red)

```

```

# set font properties
font = cv.FONT_HERSHEY_SIMPLEX
font_scale = 0.7
color = (144, 238, 144) # light green
thickness = 2

# add text to each frame when display on window
cv.putText(frame, 'Original', (10, 30), font, font_scale, color, thickness)
cv.putText(res_blue, 'Blue Detection', (10, 30), font, font_scale, color, thickness)
cv.putText(res_green, 'Green Detection', (10, 30), font, font_scale, color, thickness)
cv.putText(res_red, 'Red Detection', (10, 30), font, font_scale, color, thickness)

# combine frames to display on 1 window
combined_frame = np.hstack((frame, res_red, res_green, res_blue))

# get height and width of the combined frame
height, width, _ = combined_frame.shape

# Set scaling factor to reduce frame size
scale_factor = 0.7
new_width = int(width * scale_factor)
new_height = int(height * scale_factor)

# resize the frame for suitable window
resized_combined_frame = cv.resize(combined_frame, (new_width, new_height))

# display the output
cv.imshow('Color Detection', resized_combined_frame)

# if the key press for 5 milliseconds
k = cv.waitKey(5) & 0xFF
# esc key = ASCII 27
if k == 27:
    break

# release the camera resource
cap.release()
cv.destroyAllWindows()

if __name__ == "__main__":
    detect_target_colors()

```

A.3 Preliminary Work 3 Codes

A.3.1 Import Libraries

```
from datasets import load_dataset
```



```

from pyzbar.pyzbar import decode as zbar_decode
import cv2
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

```

A.3.2 Perspective Correction Function

```

def perspective_correction(image):
    detector = cv2.QRCodeDetector()
    retval, points = detector.detect(image) # detect QR code corners

    if retval:
        points = points[0] # get the 4 corner points
        # estimate the largest side length of the QR code
        side = max(
            np.linalg.norm(points[0] - points[1]),
            np.linalg.norm(points[1] - points[2]),
            np.linalg.norm(points[2] - points[3]),
            np.linalg.norm(points[3] - points[0])
        )
        # define square destination coordinates
        dst_points = np.array([
            [0, 0],
            [side - 1, 0],
            [side - 1, side - 1],
            [0, side - 1]
        ], dtype="float32")

        # apply perspective transformation
        M = cv2.getPerspectiveTransform(points, dst_points)
        warped = cv2.warpPerspective(image, M, (int(side), int(side)))
        return warped
    else:
        return image # no QR detected, return original

```

A.3.3 Success Rates of ZBar and OpenCV Calculation

```

total = 0
zbar_success = 0
opencv_success = 0

# Iterate through dataset images
for item in tqdm(dataset['train'], desc="Processing images"):
    img_pil = item['image'] # PIL Image
    gray = cv2.cvtColor(np.array(img_pil), cv2.COLOR_BGR2GRAY)

    # Apply perspective correction

```

```

corrected = perspective_correction(gray)

# Upscale image for better decoding
upscale = cv2.resize(corrected, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)

# Thresholding for high contrast
_, thresh = cv2.threshold(upscale, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# ----- ZBar decoding -----
zbar_results = zbar_decode(thresh)
if zbar_results:
    zbar_success += 1
else:
    # Try fallback images if thresholded fails
    zbar_results = zbar_decode(upscale)
    if zbar_results:
        zbar_success += 1
    else:
        zbar_results = zbar_decode(gray)
        if zbar_results:
            zbar_success += 1

# ----- OpenCV decoding -----
detector = cv2.QRCodeDetector()
data, points, _ = detector.detectAndDecode(thresh)
if data:
    opencv_success += 1
else:
    # Try fallback images
    data, points, _ = detector.detectAndDecode(upscale)
    if data:
        opencv_success += 1
    else:
        data, points, _ = detector.detectAndDecode(gray)
        if data:
            opencv_success += 1

total += 1

# Print final results
print(f"\nTotal images tested: {total}")
print(f"ZBar success rate: {zbar_success/total:.2%}")
print(f"OpenCV success rate: {opencv_success/total:.2%}")

```

A.3.4 QR Code Decoding Success Count by Preprocessing Method (Threshold, Upscale, Grayscale)

```

total = 0
zbar_success = 0
thresh_count = 0
upscale_count = 0
gray_count = 0

# Loop through training images
for item in tqdm(dataset['train'], desc="Processing images"):
    img_pil = item['image'] # PIL image

    gray = cv2.cvtColor(np.array(img_pil), cv2.COLOR_BGR2GRAY) # grayscale
    corrected = perspective_correction(gray) # fix perspective
    upscale = cv2.resize(corrected, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC) # enlarge
    _, thresh = cv2.threshold(upscale, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU) #
    binarize

    # Try decoding in 3 stages
    zbar_results = zbar_decode(thresh) # 1) thresholded
    if zbar_results:
        zbar_success += 1
        thresh_count += 1
    else:
        zbar_results = zbar_decode(upscale) # 2) upscaled
        if zbar_results:
            zbar_success += 1
            upscale_count += 1
        else:
            zbar_results = zbar_decode(gray) # 3) grayscale
            if zbar_results:
                zbar_success += 1
                gray_count += 1

    total += 1 # count images

# Summary
print(f"\nTotal images tested: {total}")
print(f"ZBar success count: {zbar_success}")
print(f"Thresholded decoded count: {thresh_count}")
print(f"Unscaled decoded count: {upscale_count}")
print(f"Grayscale decoded count: {gray_count}")

```

A.3.5 QR Code Decoding on 1 Image

```

def print_result(zbar_results):
    # Print decoded QR code content
    for result in zbar_results:
        print(f"  Data: {result.data.decode('utf-8')}")
        print(f"  Type: {result.type}")

```

```

# Pick sample image
index = 344
img_pil = dataset['train'][index]['image']

# Preprocessing stages
original = cv2.cvtColor(np.array(img_pil), cv2.COLOR_RGB2BGR) # PIL → OpenCV BGR
gray = cv2.cvtColor(original, cv2.COLOR_BGR2GRAY) # grayscale
corrected = perspective_correction(gray) # perspective correction
upscaled = cv2.resize(corrected, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC) # enlarge
_, thresh = cv2.threshold(upscaled, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU) #
binarize

# Show all preprocessing steps
titles = ['Original', 'Grayscale', 'Perspective Corrected', 'Upscaled', 'Thresholded']
images = [original, gray, corrected, upscaled, thresh]

plt.figure(figsize=(14, 4))
for i, img in enumerate(images):
    plt.subplot(1, len(images), i + 1)
    if len(img.shape) == 3:
        plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    else:
        plt.imshow(img, cmap='gray')
    plt.title(titles[i], fontsize=8)
    plt.axis('off')
plt.tight_layout()
plt.show()

# Try decoding with ZBar (different stages)
try:
    zbar_results = zbar_decode(thresh) # thresholded
    if zbar_results:
        print(f"\n ZBar detected {len(zbar_results)} QR code(s) in thresholded image")
        print_result(zbar_results)
    else:
        zbar_results = zbar_decode(upscaled) # upscaled
        if zbar_results:
            print(f"\n ZBar detected {len(zbar_results)} QR code(s) in upscaled image")
            print_result(zbar_results)
        else:
            zbar_results = zbar_decode(gray) # grayscale
            if zbar_results:
                print(f"\n ZBar detected {len(zbar_results)} QR code(s) in grayscale image")
                print_result(zbar_results)
            else:
                print(f"\n ZBar: No QR code detected")
except ImportError:

```

```
print(f"    ZBar not available (install with: pip install pyzbar)")
```

A.3.6 QR Code Decoding on Dataset

```
def print_result(zbar_results):
    for result in zbar_results:
        print(f"    Data: {result.data.decode('utf-8')}")
        print(f"    Type: {result.type}")

for item in tqdm(dataset['train'], desc="Processing images"):
    img_pil = item['image'] # Load PIL image

    # Convert image: PIL → NumPy → BGR
    original = cv2.cvtColor(np.array(img_pil), cv2.COLOR_RGB2BGR)

    # Preprocessing pipeline
    # Step 1: Grayscale
    gray = cv2.cvtColor(original, cv2.COLOR_BGR2GRAY)
    # Step 2: Perspective correction
    corrected = perspective_correction(gray)
    # Step 3: Upscale
    upscaled = cv2.resize(corrected, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
    # Step 4: Threshold
    _, thresh = cv2.threshold(upscaled, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    # Show processing stages
    titles = ['Original', 'Grayscale', 'Perspective Corrected', 'Upscaled', 'Thresholded']
    images = [original, gray, corrected, upscaled, thresh]
    plt.figure(figsize=(14, 4))
    for i, img in enumerate(images):
        plt.subplot(1, len(images), i + 1)
        if len(img.shape) == 3: # Color
            plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        else: # Grayscale
            plt.imshow(img, cmap='gray')
        plt.title(titles[i], fontsize=8)
        plt.axis('off')
    plt.tight_layout()
    plt.show()

    # QR decoding using ZBar (try different images if needed)
    try:
        zbar_results = zbar_decode(thresh)
        if zbar_results:
            print(f"\n    ZBar detected {len(zbar_results)} QR code(s) in thresholded image")
            print_result(zbar_results)
```

```

else:
    zbar_results = zbar_decode(upscaled)
    if zbar_results:
        print(f"\n    ZBar detected {len(zbar_results)} QR code(s) in upscaled image")
        print_result(zbar_results)
    else:
        zbar_results = zbar_decode(gray)
        if zbar_results:
            print(f"\n    ZBar detected {len(zbar_results)} QR code(s) in gray image")
            print_result(zbar_results)
        else:
            print(f"\n    ZBar: No QR code detected")
except ImportError:
    print(f"    ZBar not available (install with: pip install
pyzbar)")

```

A.4 Preliminary Work 4 Codes

A.4.1 Import Libraries

```

from typing import Dict, List, Optional, Set
from dataclasses import dataclass, field
from enum import Enum
import math, time, heapq

```

A.4.2 Defining the Class

```

# Domain model
class RoomType(Enum):
    CLASSROOM = "classroom"
    LABORATORY = "laboratory"
    TOILET = "toilet"
    STAIRCASE = "staircase"
    CORRIDOR = "corridor"
    ENTRANCE = "entrance"

@dataclass(frozen=True)
class Point:
    x: float
    y: float
    def distance_to(self, other: "Point") -> float:
        return math.hypot(self.x - other.x, self.y - other.y)
    def manhattan_distance_to(self, other: "Point") -> float:
        return abs(self.x - other.x) + abs(self.y - other.y)

@dataclass
class Rectangle:

```

```

x: float; y: float; width: float; height: float
def contains_point(self, p: Point) -> bool:
    return (self.x <= p.x <= self.x + self.width and
            self.y <= p.y <= self.y + self.height)
def center(self) -> Point:
    return Point(self.x + self.width/2, self.y + self.height/2)
def intersects(self, o: "Rectangle") -> bool:
    return not (self.x + self.width < o.x or o.x + o.width < self.x or
                self.y + self.height < o.y or o.y + o.height < self.y)

@dataclass
class Room:
    id: str
    name: str
    room_type: RoomType
    center: Point
    bounds: Rectangle
    connections: Set[str] = field(default_factory=set)
    capacity: Optional[int] = None
    floor: str = "ground"
    description: Optional[str] = None
    access_points: List[Point] = field(default_factory=list)
    def add_connection(self, room_id: str):
        self.connections.add(room_id)

@dataclass
class Connection:
    from_room: str
    to_room: str
    distance: float
    pathway_type: str = "corridor"
    waypoints: List[Point] = field(default_factory=list)
    is_accessible: bool = True
    width: float = 2.0

@dataclass
class PathResult:
    path: List[str]
    total_distance: float
    waypoints: List[Point]
    found: bool
    execution_time: float

@dataclass(order=True)
class _PQEntry:
    f: float
    h: float
    _counter: int

```

```

node_id: str = field(compare=False)

@dataclass
class _Node:
    room_id: str
    g: float
    h: float
    parent: Optional[str]

```

A.4.3 Defining the Map and Creating A* Algorithm

```

# Map + A*
class CoordinateMapSystem:
    def __init__(self, building_name: str, floor_name: str):
        self.building_name = building_name
        self.floor_name = floor_name
        self.rooms: Dict[str, Room] = {}
        self.connections: Dict[str, List[Connection]] = {}
        self._push_counter = 0

    def add_room(self, room: Room):
        self.rooms[room.id] = room
        self.connections.setdefault(room.id, [])

    def add_connection(self, c: Connection):
        self.connections.setdefault(c.from_room, []).append(c)
        rev = Connection(
            from_room=c.to_room, to_room=c.from_room, distance=c.distance,
            pathway_type=c.pathway_type,
            waypoints=list(reversed(c.waypoints)) if c.waypoints else [],
            is_accessible=c.is_accessible, width=c.width
        )
        self.connections.setdefault(c.to_room, []).append(rev)
        self.rooms[c.from_room].add_connection(c.to_room)
        self.rooms[c.to_room].add_connection(c.from_room)

    def get_neighbors(self, room_id: str) -> List[str]:
        return [c.to_room for c in self.connections.get(room_id, []) if c.is_accessible]

    def get_connection(self, a: str, b: str) -> Optional[Connection]:
        for c in self.connections.get(a, []):
            if c.to_room == b:
                return c
        return None

    def heuristic(self, a: str, b: str) -> float:
        return self.rooms[a].center.distance_to(self.rooms[b].center)

```



```

def connection_length(self, conn: Connection) -> float:
    if "CORRIDOR" in conn.from_room or "CORRIDOR" in conn.to_room:
        return conn.distance
    pts = [self.rooms[conn.from_room].center] + conn.waypoints +
    [self.rooms[conn.to_room].center]
    return sum(pts[i].distance_to(pts[i+1]) for i in range(len(pts)-1))

def segment_cost(self, conn: Connection) -> float:
    return self.connection_length(conn)

def _stitch_waypoints(self, path_ids: List[str]) -> List[Point]:
    if not path_ids: return []
    pts: List[Point] = [self.rooms[path_ids[0]].center]
    for a, b in zip(path_ids, path_ids[1:]):
        c = self.get_connection(a, b)
        if not c: continue
        pts.extend(c.waypoints)
        pts.append(self.rooms[b].center)
    cleaned = [pts[0]]
    for p in pts[1:]:
        if cleaned[-1].distance_to(p) > 0.05:
            cleaned.append(p)
    return cleaned

def a_star_pathfind(self, start_room: str, goal_room: str) -> PathResult:
    t0 = time.time()
    if start_room not in self.rooms or goal_room not in self.rooms:
        return PathResult([], 0.0, [], False, time.time()-t0)
    if start_room == goal_room:
        return PathResult([start_room], 0.0, [self.rooms[start_room].center], True, time.time()-t0)

    open_heap: List[_PQEntry] = []
    open_map: Dict[str, _Node] = {}
    closed: Set[str] = set()

    h0 = self.heuristic(start_room, goal_room)
    open_map[start_room] = _Node(start_room, 0.0, h0, None)
    self._push_counter += 1
    heapq.heappush(open_heap, _PQEntry(f=h0, h=h0, _counter=self._push_counter,
    node_id=start_room))

    while open_heap:
        u_id = heapq.heappop(open_heap).node_id
        if u_id in closed:
            continue
        u = open_map[u_id]
        closed.add(u_id)

```

```

if u_id == goal_room:
    path: List[str] = []
    cur = u_id
    while cur is not None:
        path.append(cur)
        cur = open_map[cur].parent
    path.reverse()
    return PathResult(path, u.g, self._stitch_waypoints(path), True, time.time()-t0)

for v_id in self.get_neighbors(u_id):
    if v_id in closed:
        continue
    conn = self.get_connection(u_id, v_id)
    if not conn or not conn.is_accessible:
        continue
    tentative_g = u.g + self.segment_cost(conn)
    prev = open_map.get(v_id)
    if prev is None or tentative_g < prev.g - 1e-12:
        h = self.heuristic(v_id, goal_room)
        open_map[v_id] = _Node(v_id, tentative_g, h, u_id)
        f = tentative_g + h
        self._push_counter += 1
        heapq.heappush(open_heap, _PQEntry(f=f, h=h, _counter=self._push_counter,
node_id=v_id))

return PathResult([], 0.0, [], False, time.time()-t0)

def find_shortest_path(self, start_room: str, goal_room: str, verbose: bool = False) -> PathResult:
    sid = self._resolve_room_id(start_room)
    gid = self._resolve_room_id(goal_room)
    if not sid or not gid:
        return PathResult([], 0.0, [], False, 0.0)
    return self.a_star_pathfind(sid, gid)

def _resolve_room_id(self, ident: str) -> Optional[str]:
    if ident in self.rooms: return ident
    for rid, r in self.rooms.items():
        if r.name.lower() == ident.lower():
            return rid
    return None

```

A.4.4 Connect The Map Coordinate

```

# Map Coordinate
def create_fict_map() -> CoordinateMapSystem:
    m = CoordinateMapSystem("FICT Building", "Ground Floor")
    x0, dx = 20, 25
    room_w, room_h = 20, 15

```

```

y_top, y_bot = 80, 40
toilet_w = 8

# ---- ALL ROOMS
rooms_data = [
    # Top row classrooms
    ("N007", "Open-Office Style Classroom", RoomType.CLASSROOM, Point(x0,
y_top), Rectangle(x0-10, 72, room_w, room_h), [Point(x0,72)]),
    ("N006", "Classroom N006", RoomType.CLASSROOM, Point(x0+dx,
y_top), Rectangle(x0+dx-10, 72, room_w, room_h), [Point(x0+dx,72)]),
    ("N005", "Classroom N005", RoomType.CLASSROOM, Point(x0+2*dx,
y_top), Rectangle(x0+2*dx-10, 72, room_w, room_h), [Point(x0+2*dx,72)]),
    ("N004", "Classroom N004", RoomType.CLASSROOM, Point(x0+3*dx,
y_top), Rectangle(x0+3*dx-10, 72, room_w, room_h), [Point(x0+3*dx,72)]),
    ("N003", "Classroom N003", RoomType.CLASSROOM, Point(x0+4*dx,
y_top), Rectangle(x0+4*dx-10, 72, room_w, room_h), [Point(x0+4*dx,72)]),
    ("N002", "Classroom N002", RoomType.CLASSROOM, Point(x0+5*dx,
y_top), Rectangle(x0+5*dx-10, 72, room_w, room_h), [Point(x0+5*dx,72)]),
    ("N001", "Classroom N001", RoomType.CLASSROOM, Point(x0+6*dx,
y_top), Rectangle(x0+6*dx-10, 72, room_w, room_h), [Point(x0+6*dx,72)]),

    # Bottom row labs & toilets
    ("N008", "Microsoft Software Engineering Laboratory", RoomType.LABORATORY, Point(x0,
y_bot), Rectangle(x0-10, 25, room_w, room_h), [Point(x0,55)]),
    ("NGT3", "Disabled Toilet", RoomType.TOILET, Point(x0+dx,
y_bot), Rectangle(x0+dx-4, 32, toilet_w, toilet_w), [Point(x0+dx,50)]),
    ("NGT4", "Male Toilet", RoomType.TOILET, Point(x0+2*dx,
y_bot), Rectangle(x0+2*dx-4, 32, toilet_w, toilet_w), [Point(x0+2*dx,50)]),
    ("NGT5", "Female Toilet", RoomType.TOILET, Point(x0+2.4*dx,
y_bot), Rectangle(x0+2.4*dx-4, 32, toilet_w, toilet_w), [Point(x0+2.4*dx,50)]),
    ("N009", "Silverlab", RoomType.LABORATORY, Point(x0+3*dx,
y_bot), Rectangle(x0+3*dx-10, 25, room_w, room_h), [Point(x0+3*dx,55)]),
    ("N010", "Cisco Networking Academy Laboratory", RoomType.LABORATORY,
Point(x0+4*dx, y_bot), Rectangle(x0+4*dx-10, 25, room_w, room_h), [Point(x0+4*dx,55)]),
    ("N011", "IPSR Lab", RoomType.LABORATORY, Point(x0+5*dx,
y_bot), Rectangle(x0+5*dx-10, 25, room_w, room_h), [Point(x0+5*dx,55)]),
    ("N012", "Lab N012", RoomType.LABORATORY, Point(x0+6*dx,
y_bot), Rectangle(x0+6*dx-10, 25, room_w, room_h), [Point(x0+6*dx,55)]),
    ("NGT1", "Female Toilet", RoomType.TOILET, Point(x0+7*dx,
y_bot), Rectangle(x0+7*dx-4, 32, toilet_w, toilet_w), [Point(x0+7*dx,50)]),
    ("NGT2", "Male Toilet", RoomType.TOILET, Point(x0+7.4*dx,
y_bot), Rectangle(x0+7.4*dx-4, 32, toilet_w, toilet_w), [Point(x0+7.4*dx,50)]),

    # Entrance and corridors
    ("ENTRANCE_MAIN", "Main Entrance", RoomType.ENTRANCE, Point(x0+3*dx, 90),
Rectangle(x0+3*dx-5, 87, 10, 6), []),
    ("CORRIDOR_JUNCTION_MID", "Main Corridor Junction", RoomType.CORRIDOR,
Point(x0+3*dx, 65), Rectangle(x0+3*dx-2, 63, 4, 4), []),

```

```

    ("CORRIDOR_JUNCTION_TOP", "Top Horizontal Corridor", RoomType.CORRIDOR,
    Point(x0+3*dx, 85), Rectangle(x0-10, 83, 180, 4), []), # Horizontal corridor in front of N007-N001
    ("CORRIDOR_JUNCTION_BOTTOM", "Bottom Horizontal Corridor",
    RoomType.CORRIDOR, Point(x0+3*dx, 45), Rectangle(x0-10, 43, 205, 4), []), # Horizontal
    corridor in front of N008-NGT2
    ("LEFT_CORRIDOR", "Left Corridor", RoomType.CORRIDOR, Point(x0-5, 65),
    Rectangle(x0-7, 63, 4, 4), []),
    ("RIGHT_CORRIDOR", "Right Corridor", RoomType.CORRIDOR, Point(x0+7*dx, 65),
    Rectangle(x0+7*dx-2, 63, 4, 4), []),
    ]

    for rid, name, rtype, center, bounds, aps in rooms_data:
        m.add_room(Room(id=rid, name=name, room_type=rtype, center=center, bounds=bounds,
            access_points=aps, capacity=30 if rtype in (RoomType.LABORATORY,
            RoomType.CLASSROOM) else None))

    # ---- FIXED CONNECTIONS with geometrically accurate distances
    conns = [
        # Horizontal connections - Top row (through top horizontal corridor)
        ("N007", "CORRIDOR_JUNCTION_TOP", 10.0, "corridor"),
        ("CORRIDOR_JUNCTION_TOP", "N007", 10.0, "corridor"),
        ("N006", "CORRIDOR_JUNCTION_TOP", 10.0, "corridor"),
        ("CORRIDOR_JUNCTION_TOP", "N006", 10.0, "corridor"),
        ("N005", "CORRIDOR_JUNCTION_TOP", 10.0, "corridor"),
        ("CORRIDOR_JUNCTION_TOP", "N005", 10.0, "corridor"),
        ("N004", "CORRIDOR_JUNCTION_TOP", 10.0, "corridor"),
        ("CORRIDOR_JUNCTION_TOP", "N004", 10.0, "corridor"),
        ("N003", "CORRIDOR_JUNCTION_TOP", 10.0, "corridor"),
        ("CORRIDOR_JUNCTION_TOP", "N003", 10.0, "corridor"),
        ("N002", "CORRIDOR_JUNCTION_TOP", 10.0, "corridor"),
        ("CORRIDOR_JUNCTION_TOP", "N002", 10.0, "corridor"),
        ("N001", "CORRIDOR_JUNCTION_TOP", 10.0, "corridor"),
        ("CORRIDOR_JUNCTION_TOP", "N001", 10.0, "corridor"),

        # Horizontal connections - Bottom row (through bottom horizontal corridor)
        ("N008", "CORRIDOR_JUNCTION_BOTTOM", 15.0, "corridor"),
        ("CORRIDOR_JUNCTION_BOTTOM", "N008", 15.0, "corridor"),
        ("NGT3", "CORRIDOR_JUNCTION_BOTTOM", 15.0, "corridor"),
        ("CORRIDOR_JUNCTION_BOTTOM", "NGT3", 15.0, "corridor"),
        ("NGT4", "CORRIDOR_JUNCTION_BOTTOM", 15.0, "corridor"),
        ("CORRIDOR_JUNCTION_BOTTOM", "NGT4", 15.0, "corridor"),
        ("NGT5", "CORRIDOR_JUNCTION_BOTTOM", 15.0, "corridor"),
        ("CORRIDOR_JUNCTION_BOTTOM", "NGT5", 15.0, "corridor"),
        ("N009", "CORRIDOR_JUNCTION_BOTTOM", 15.0, "corridor"),
        ("CORRIDOR_JUNCTION_BOTTOM", "N009", 15.0, "corridor"),
        ("N010", "CORRIDOR_JUNCTION_BOTTOM", 15.0, "corridor"), # FIXED: N010 connects to
        bottom corridor
        ("CORRIDOR_JUNCTION_BOTTOM", "N010", 15.0, "corridor"),
    ]

```

```

("N011","CORRIDOR_JUNCTION_BOTTOM",15.0,"corridor"),
("CORRIDOR_JUNCTION_BOTTOM","N011",15.0,"corridor"),
("N012","CORRIDOR_JUNCTION_BOTTOM",15.0,"corridor"),
("CORRIDOR_JUNCTION_BOTTOM","N012",15.0,"corridor"),
("NGT1","CORRIDOR_JUNCTION_BOTTOM",15.0,"corridor"),
("CORRIDOR_JUNCTION_BOTTOM","NGT1",15.0,"corridor"),
("NGT2","CORRIDOR_JUNCTION_BOTTOM",15.0,"corridor"),
("CORRIDOR_JUNCTION_BOTTOM","NGT2",15.0,"corridor"),

# FIXED: Direct connections between adjacent rooms on same level
("N009","N010",25.0,"corridor"), # N009 and N010 are adjacent
("N010","N011",25.0,"corridor"), # N010 and N011 are adjacent
("N011","N012",25.0,"corridor"), # N011 and N012 are adjacent

# Main vertical connections
("ENTRANCE_MAIN","CORRIDOR_JUNCTION_TOP",5.0,"entrance"), # Entrance
connects to top
("CORRIDOR_JUNCTION_TOP","CORRIDOR_JUNCTION_MID",20.0,"corridor"),
("CORRIDOR_JUNCTION_MID","CORRIDOR_JUNCTION_BOTTOM",20.0,"corridor"),

# Side corridors
("CORRIDOR_JUNCTION_MID","LEFT_CORRIDOR",80.0,"corridor"),
("CORRIDOR_JUNCTION_MID","RIGHT_CORRIDOR",100.0,"corridor"),

("LEFT_CORRIDOR","CORRIDOR_JUNCTION_TOP",30.0,"corridor"), # Left corridor
connects to both levels
("LEFT_CORRIDOR","CORRIDOR_JUNCTION_BOTTOM",30.0,"corridor"),

("RIGHT_CORRIDOR","CORRIDOR_JUNCTION_TOP",15.0,"corridor"), # Right corridor
connects to both levels
("RIGHT_CORRIDOR","CORRIDOR_JUNCTION_BOTTOM",15.0,"corridor"),
]

for a,b,d,kind in conns:
    m.add_connection(Connection(from_room=a, to_room=b, distance=d, pathway_type=kind))

return m

```

A.4.5 Output Functions

```

# Output functions
def print_all_objects(fict_map: CoordinateMapSystem):
    """Output 1: ALL OBJECTS IN MAP with adjacent objects"""
    print("=" * 80)
    print("ALL OBJECTS IN MAP")
    print("=" * 80)

    # Sort rooms for consistent output - by room type, then by ID

```

```

def sort_key_for_output(room):
    # Priority order: corridors first, then entrance, then others by ID
    type_priority = {
        RoomType.CORRIDOR: 0,
        RoomType.ENTRANCE: 1,
        RoomType.CLASSROOM: 2,
        RoomType.LABORATORY: 3,
        RoomType.TOILET: 4
    }
    return (type_priority.get(room.room_type, 999), room.id)

sorted_rooms = sorted(fict_map.rooms.values(), key=sort_key_for_output)

# First show ALL objects with basic info
for room in sorted_rooms:
    x, y = int(room.center.x), int(room.center.y)
    print(f"Object: {room.name} ({room.id}) | Type: {room.room_type.value} | Coordinates: ({x}, {y})")

print()
print("=" * 80)
print("CLASSROOM & NEIGHBORS LAYOUT")
print("=" * 80)

def find_adjacent_objects(room_id: str, room_center: Point):
    """Find objects directly to the left and right of this room"""
    # Get ALL rooms on roughly the same Y level (within 10 units) - including the current room
    same_level_rooms = []

    for other_id, other_room in fict_map.rooms.items():
        if (abs(other_room.center.y - room_center.y) <= 10 and
            other_room.room_type in [RoomType.CLASSROOM, RoomType.LABORATORY,
RoomType.TOILET]):
            same_level_rooms.append((other_id, other_room))

    # Sort by X coordinate
    same_level_rooms.sort(key=lambda x: x[1].center.x)

    # Find current room's position in the sorted list
    current_index = -1
    for i, (other_id, other_room) in enumerate(same_level_rooms):
        if other_id == room_id:
            current_index = i
            break

    left_neighbor = None
    right_neighbor = None

```

```

if current_index >= 0:
    # Left neighbor (previous in sorted list)
    if current_index > 0:
        left_neighbor = same_level_rooms[current_index - 1][0]
    # Right neighbor (next in sorted list)
    if current_index < len(same_level_rooms) - 1:
        right_neighbor = same_level_rooms[current_index + 1][0]

# Special cases for corridors as neighbors
current_room = fict_map.rooms[room_id]

# For N007 - beside left should be LEFT_CORRIDOR
if room_id == "N007":
    left_neighbor = "LEFT_CORRIDOR"

# For N003 and N004 - they connect to top horizontal corridor
elif room_id == "N003":
    left_neighbor = "CORRIDOR_JUNCTION_TOP" # Top horizontal corridor
elif room_id == "N004":
    right_neighbor = "CORRIDOR_JUNCTION_TOP" # Top horizontal corridor

# For N010 and N011 - they connect to bottom horizontal corridor
elif room_id == "N010":
    right_neighbor = "CORRIDOR_JUNCTION_BOTTOM" # Bottom horizontal corridor
elif room_id == "N011":
    left_neighbor = "CORRIDOR_JUNCTION_BOTTOM" # Bottom horizontal corridor

# For NGT1 and NGT2 toilets - fix the reversed logic
elif room_id == "NGT1":
    left_neighbor = "NGT2"
    right_neighbor = "RIGHT_CORRIDOR"
elif room_id == "NGT2":
    left_neighbor = "N012"
    right_neighbor = "NGT1"

return left_neighbor, right_neighbor

def find_opposite_room(room_id: str, room_center: Point):
    """Find the room(s) opposite to this room"""
    current_room = fict_map.rooms[room_id]

    opposite_mappings = {
        "N001": ["NGT1", "NGT2"], # N001 opposite is NGT1,NGT2
        "N002": ["N012"],        # N002 opposite is N012
        "N003": ["N011"],        # N003 opposite is IPSR Lab (N011)
        "N004": ["N010"],        # N004 opposite is Cisco (N010)
        "N005": ["N009"],        # N005 opposite is Silverlab (N009)
        "N006": ["NGT3", "NGT4", "NGT5"], # N006 opposite has 3 toilets
    }

```

```

        "N007": ["N008"],      # N007 opposite is N008
        "N008": ["N007"],      # N008 opposite is N007
        "N009": ["N005"],      # N009 opposite is N005
        "N010": ["N004"],      # N010 opposite is N004
        "N011": ["N003"],      # N011 opposite is N003
        "N012": ["N002"],      # N012 opposite is N002
        "NGT1": ["N001"],      # NGT1 opposite is N001
        "NGT2": ["N001"],      # NGT2 opposite is N001
        "NGT3": ["N006"],      # NGT3 opposite is N006
        "NGT4": ["N006"],      # NGT4 opposite is N006
        "NGT5": ["N006"],      # NGT5 opposite is N006
    }

    opposite_ids = opposite_mappings.get(room_id, [])

    if len(opposite_ids) == 1:
        return fict_map.rooms[opposite_ids[0]].name
    elif len(opposite_ids) > 1:
        # Multiple opposites - join their names
        names = [fict_map.rooms[oid].name for oid in opposite_ids]
        return ", ".join(names)
    else:
        return "None"

    # Sort rooms for neighbor analysis - by position (Y first, then X)
    analysis_rooms = sorted([r for r in fict_map.rooms.values()
                             if r.room_type in [RoomType.CLASSROOM, RoomType.LABORATORY,
                             RoomType.TOILET]],
                             key=lambda r: (r.center.y, r.center.x))

    # Show neighbor relationships for classrooms, labs, toilets
    for room in analysis_rooms:
        left_neighbor, right_neighbor = find_adjacent_objects(room.id, room.center)

        left_name = fict_map.rooms[left_neighbor].name if left_neighbor and left_neighbor in
fict_map.rooms else "None"
        right_name = fict_map.rooms[right_neighbor].name if right_neighbor and right_neighbor in
fict_map.rooms else "None"

        # Use the new opposite finding function
        opposite = find_opposite_room(room.id, room.center)

        print(f'({room.id}, Beside Left: {left_name}, Beside Right: {right_name}, Opposite:
{opposite})')

def print_n001_routes(fict_map: CoordinateMapSystem):
    """Output 2: A* routes from N001 to all classrooms, laboratories, and toilets"""
    start = "N001"

```



```

start_id = fict_map._resolve_room_id(start)
if not start_id:
    print("    N001 not found")
    return

# Get all target rooms (classrooms, labs, toilets except N001)
targets = [
    rid for rid, room in fict_map.rooms.items()
    if room.room_type in {RoomType.CLASSROOM, RoomType.LABORATORY,
RoomType.TOILET} and rid != start_id
]

# Sort them by type and ID
classrooms = sorted([rid for rid in targets if fict_map.rooms[rid].room_type ==
RoomType.CLASSROOM])
laboratories = sorted([rid for rid in targets if fict_map.rooms[rid].room_type ==
RoomType.LABORATORY])
toilets = sorted([rid for rid in targets if fict_map.rooms[rid].room_type == RoomType.TOILET])

# Order: classrooms first, then labs, then toilets
ordered_targets = classrooms + laboratories + toilets

print("=" * 80)
print(f"    A* routes from {start} to all classrooms, laboratories, and toilets")
print("=" * 80)
print(f"{'Dest ID':<8} {'Type':<11} {'Found':<6} {'Distance (m)':>13} Path")
print("-" * 80)

found_count = 0
for dest in ordered_targets:
    result = fict_map.find_shortest_path(start, dest, verbose=False)
    room = fict_map.rooms[dest]

    found = "yes" if result.found else "no"
    distance = f"{result.total_distance:.1f}" if result.found else "--"
    path = " → ".join(result.path) if result.found else "(no path)"

    print(f"{'dest':<8} {'room.room_type.value':<11} {'found':<6} {'distance':>13} {'path}'")

    if result.found:
        found_count += 1

print("-" * 80)
print(f"    {found_count}/{len(ordered_targets)} routes found.")

def debug_pathfinding_objects(fict_map: CoordinateMapSystem):
    """Debug function to show all objects and their connections"""
    print("=" * 80)

```

```

print("    ALL PATHFINDING OBJECTS & CONNECTIONS")
print("=" * 80)

# Sort rooms by type and ID for consistent output
def sort_key_debug(room):
    type_priority = {
        RoomType.CORRIDOR: 0,
        RoomType.ENTRANCE: 1,
        RoomType.CLASSROOM: 2,
        RoomType.LABORATORY: 3,
        RoomType.TOILET: 4
    }
    return (type_priority.get(room.room_type, 999), room.id)

sorted_rooms = sorted(fict_map.rooms.values(), key=sort_key_debug)

for room in sorted_rooms:
    x, y = int(room.center.x), int(room.center.y)
    neighbors = fict_map.get_neighbors(room.id)
    neighbor_names = [fict_map.rooms[nid].name for nid in neighbors] if neighbors else ["None"]

    print(f"    {room.name} ({room.id})")
    print(f"    Type: {room.room_type.value} | Coordinates: ({x}, {y})")
    print(f"    Connected to: {' '.join(neighbor_names)}")
    print()

```

A.4.6 Main Functions

```

def main():
    """Main function - produces debug info and the 2 required outputs"""
    try:
        # Create the map
        fict_map = create_fict_map()

        # Debug output - show all objects and connections
        debug_pathfinding_objects(fict_map)

        # Output 1: All objects
        print_all_objects(fict_map)

        print() # Empty line separator

        # Output 2: N001 routes
        print_n001_routes(fict_map)

    except Exception as e:
        print(f"    ERROR: {e}")
    import traceback

```

APPENDIX A

```
        traceback.print_exc()

if __name__ == "__main__":
    main()
```

APPENDIX B

CODING RESULTS

B.1 Preliminary Work 1 Results

```
Select QR code color to detect:
1. Red
2. Green
3. Blue
Enter choice (1-3): _
```

Figure B.1.1: Selecting QR Code Colour

```
QR target color set to RED
SYSTEM: QR code color set to RED
SYSTEM: System is now active and scanning. You can move around freely.
SYSTEM: System active. Continuously scanning for QR codes. Rotate slowly to look around.
```

Figure B.1.2: Confirming QR Code Colour

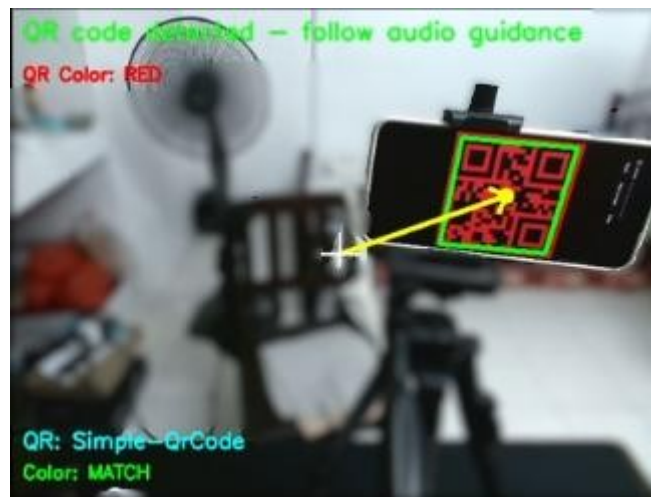


Figure B.1.3: QR Code Detected

```
DIRECTION: Turn left to find the QR code
```

Figure B.1.4: Guiding User Towards the QR Code

```
SYSTEM: QR code detected: Simple-QrCode
DIRECTION: QR code content: Simple-QrCode. Please continue to the next waypoint.
```

Figure B.1.5: QR Code Has Been Read



Figure B.1.6: Wrong QR Code Colour

B.2 Preliminary Work 2 Results

Under low light condition:



Figure B.2.1: QR Code Detection at 50 cm (81 cm², Low Light)

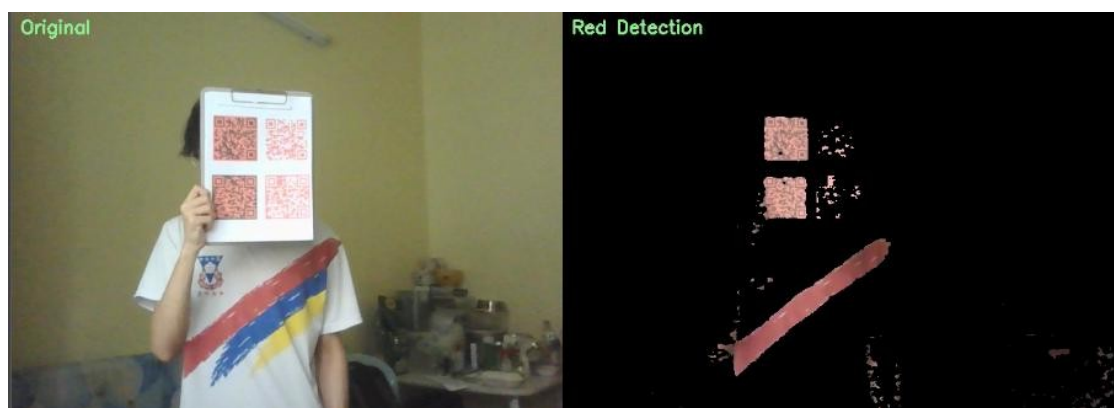


Figure B.2.2: QR Code Detection at 100 cm (81 cm², Low Light)

APPENDIX B

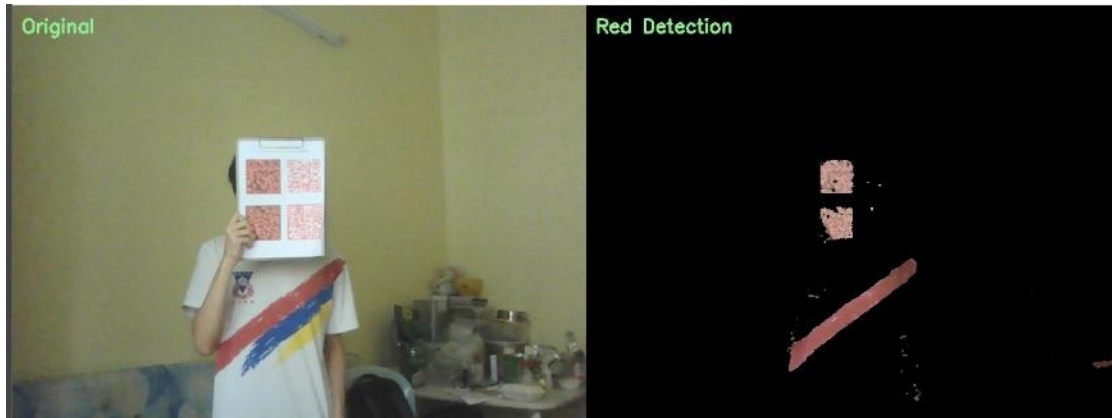


Figure B.2.3: QR Code Detection at 150 cm (81 cm², Low Light)



Figure B.2.4: QR Code Detection at 50 cm (Red Module + White Background, 324 cm², Low Light)



Figure B.2.5: QR Code Detection at 100 cm (Red Module + White Background, 324 cm², Low Light)

APPENDIX B

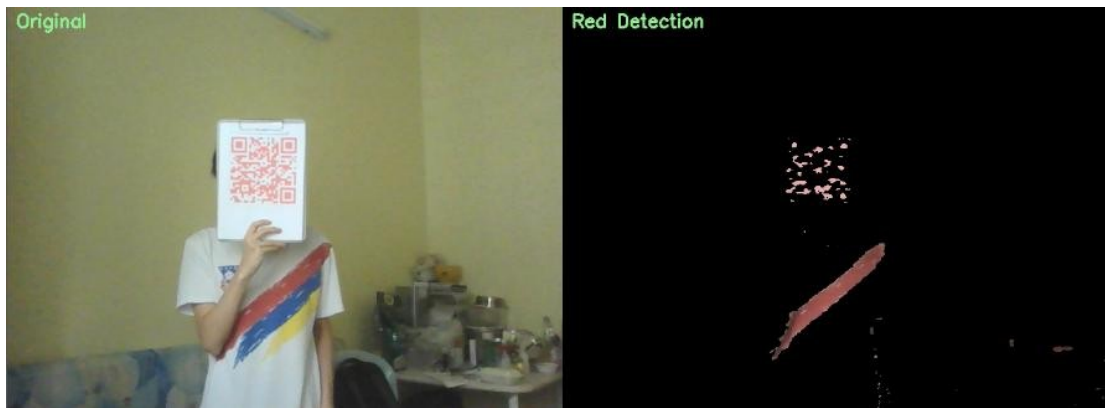


Figure B.2.6: QR Code Detection at 150 cm (Red Module + White Background, 324 cm², Low Light)



Figure B.2.7: QR Code Detection at 50 cm (White Module + Red Background, 324 cm², Low Light)

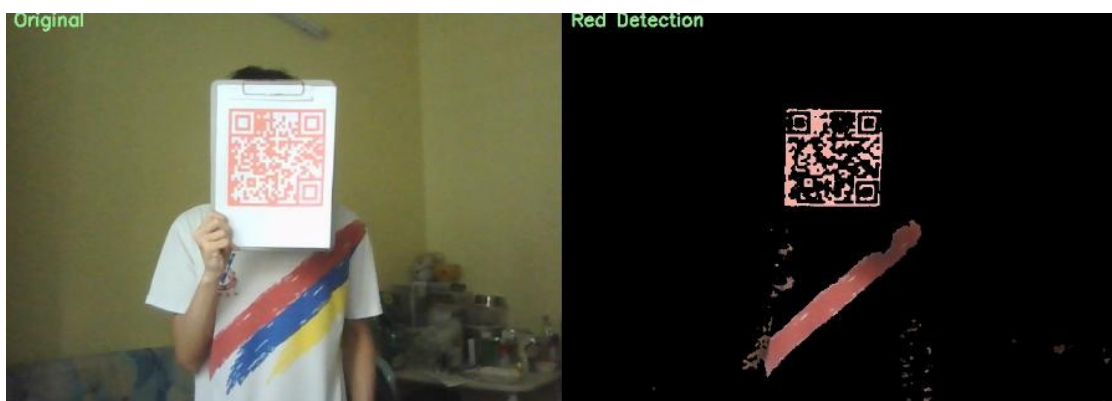


Figure B.2.8: QR Code Detection at 100 cm (White Module + Red Background, 324 cm², Low Light)

APPENDIX B



Figure B.2.9: QR Code Detection at 150 cm (White Module + Red Background, 324 cm², Low Light)



Figure B.2.10: QR Code Detection at 50 cm (Red Module + Black Background, 324 cm², Low Light)



Figure B.2.11: QR Code Detection at 100 cm (Red Module + Black Background, 324 cm², Low Light)



Figure B.2.12: QR Code Detection at 150 cm (Red Module + Black Background, 324 cm², Low Light)



Figure B.2.13: QR Code Detection at 50 cm (Black Module + Red Background, 324 cm², Low Light)



Figure B.2.14: QR Code Detection at 100 cm (Black Module + Red Background, 324 cm², Low Light)

APPENDIX B



Figure B.2.15: QR Code Detection at 150 cm (Black Module + Red Background, 324 cm², Low Light)

Under bright lighting condition:



Figure B.2.16: QR Code Detection at 50 cm (81 cm², Bright)



Figure B.2.17: QR Code Detection at 100 cm (81 cm², Bright)

APPENDIX B

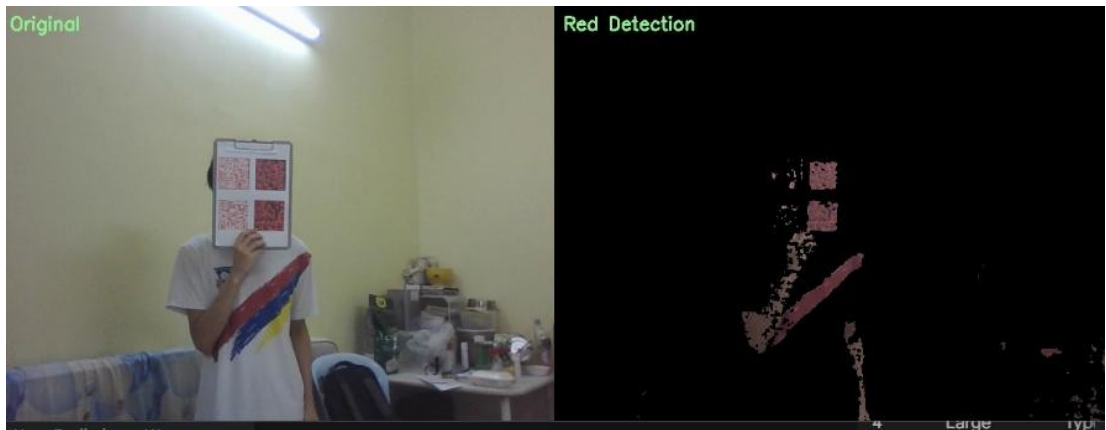


Figure B.2.18: QR Code Detection at 150 cm (81 cm², Bright)

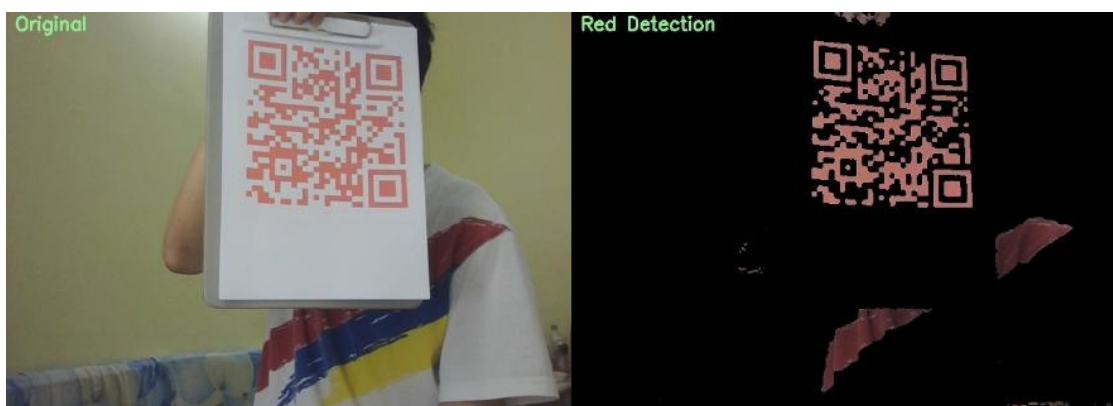


Figure B.2.19: QR Code Detection at 50 cm (Red Module + White Background, 324 cm², Bright)

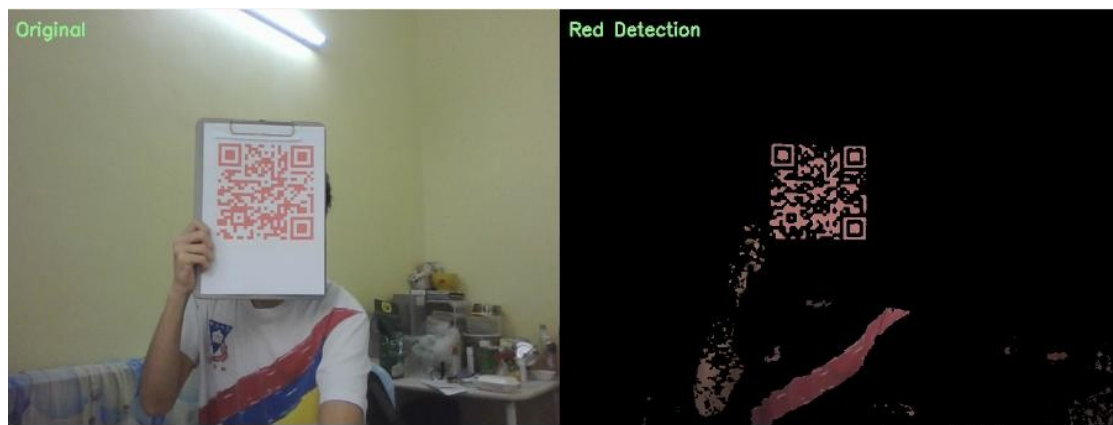


Figure B.2.20: QR Code Detection at 100 cm (Red Module + White Background, 324 cm², Bright)

APPENDIX B



Figure B.2.21: QR Code Detection at 150 cm (Red Module + White Background, 324 cm², Bright)



Figure B.2.22: QR Code Detection at 50 cm (White Module + Red Background, 324 cm², Bright)



Figure B.2.23: QR Code Detection at 100 cm (White Module + Red Background, 324 cm², Bright)

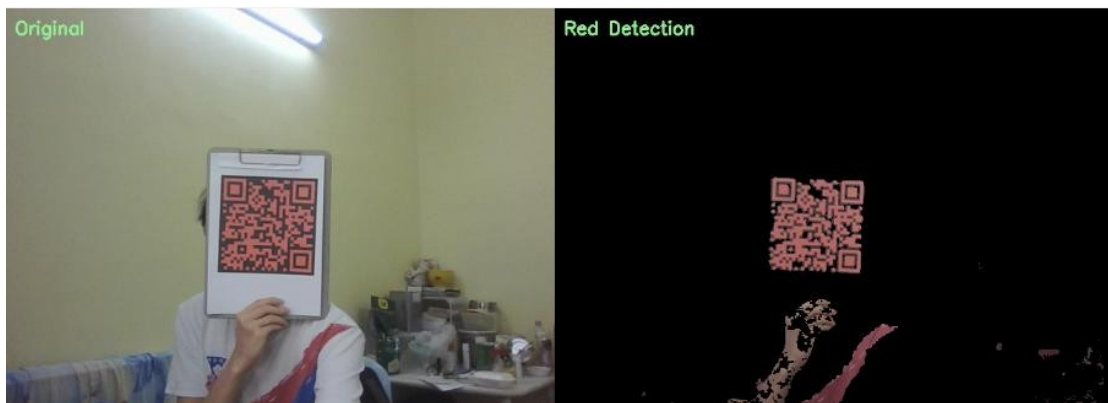
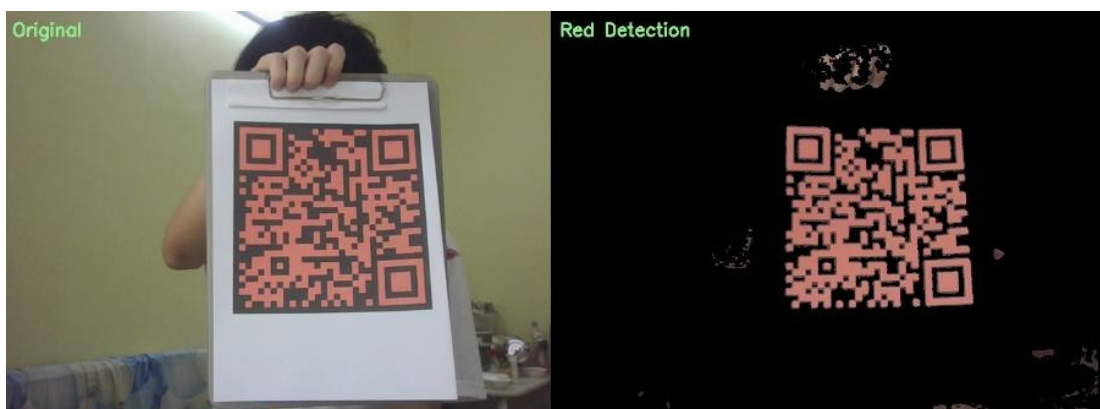




Figure B.2.27: QR Code Detection at 150 cm (Red Module + Black Background, 324 cm², Bright)

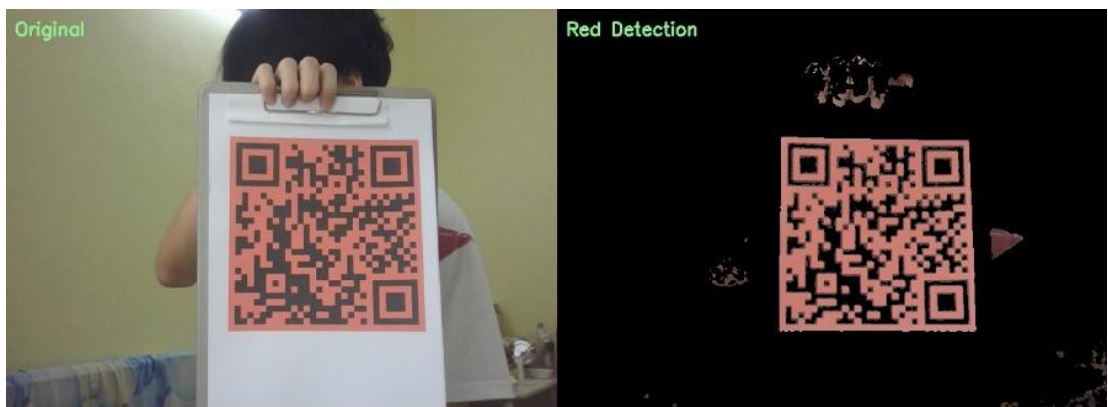


Figure B.2.28: QR Code Detection at 50 cm (Black Module + Red Background, 324 cm², Bright)

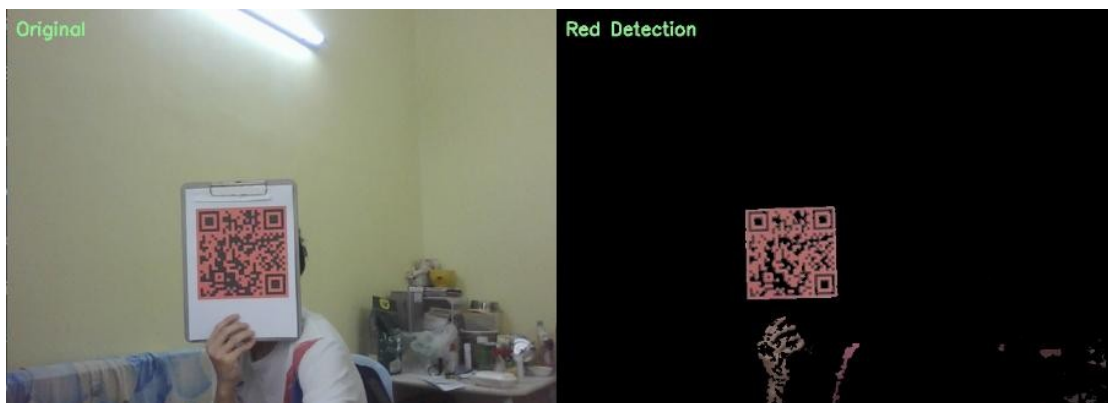


Figure B.2.29: QR Code Detection at 100 cm (Black Module + Red Background, 324 cm², Bright)



Figure B.2.30: QR Code Detection at 150 cm (Black Module + Red Background, 324 cm², Bright)

B.3 Preliminary Work 3 Results



Figure B.3.1: Decoding Success Rates of ZBar and OpenCV

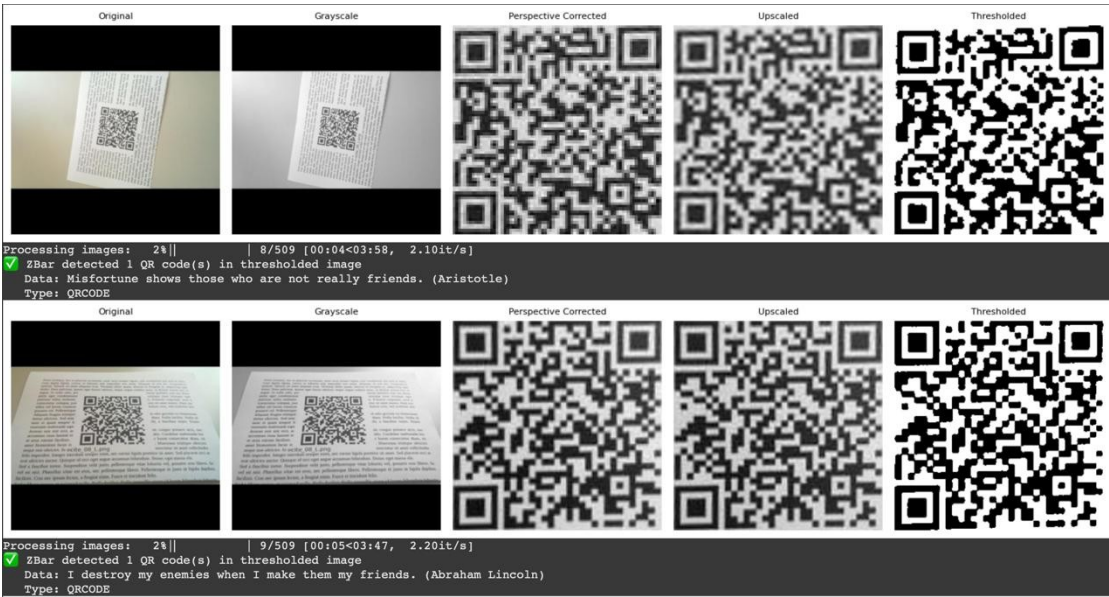


Figure B.3.2: Examples of Successfully Decoded QR Code Images



Figure B.3.3: Example of Failed Decoding Using Binarized Images



Figure B.3.4: Examples of Failed Decoding due to Perspective Correction Failure and Image Blur

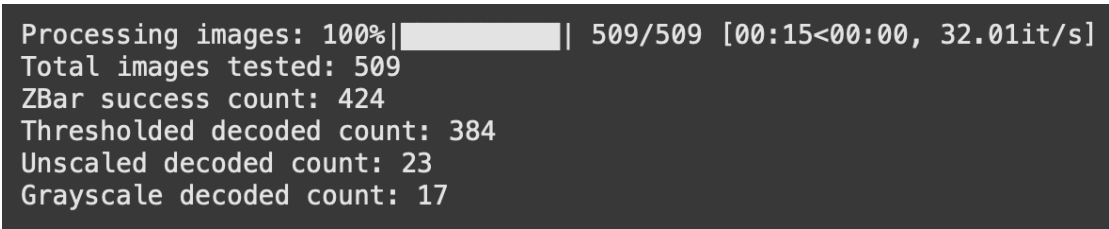


Figure B.3.5: Decoding Counts for Binarized, Upscaled, and Grayscale Images

APPENDIX B



Figure B.3.6: Examples of Failed Decoding due to Lighting Reflection



Figure B.3.7: Examples of Failed Decoding of Distorted QR Code Images



Figure B.3.8: Examples of Failed Decoding due to Shadow

B.4 Preliminary Work 4 Results

```

=====
ALL OBJECTS IN MAP
=====
Object: Bottom Junction Corridor (CORRIDOR_JUNCTION_BOTTOM) | Type: corridor | Coordinates: (95, 45)
Object: Main Corridor Junction (CORRIDOR_JUNCTION_MID) | Type: corridor | Coordinates: (95, 65)
Object: Top Junction Corridor (CORRIDOR_JUNCTION_TOP) | Type: corridor | Coordinates: (95, 85)
Object: Left Corridor (LEFT_CORRIDOR) | Type: corridor | Coordinates: (15, 65)
Object: Right Corridor (RIGHT_CORRIDOR) | Type: corridor | Coordinates: (195, 65)
Object: Main Entrance (ENTRANCE_MAIN) | Type: entrance | Coordinates: (95, 90)
Object: Classroom N001 (N001) | Type: classroom | Coordinates: (170, 80)
Object: Classroom N002 (N002) | Type: classroom | Coordinates: (145, 80)
Object: Classroom N003 (N003) | Type: classroom | Coordinates: (120, 80)
Object: Classroom N004 (N004) | Type: classroom | Coordinates: (95, 80)
Object: Classroom N005 (N005) | Type: classroom | Coordinates: (70, 80)
Object: Classroom N006 (N006) | Type: classroom | Coordinates: (45, 80)
Object: Open-Office Style Classroom (N007) | Type: classroom | Coordinates: (20, 80)
Object: Microsoft Software Engineering Laboratory (N008) | Type: laboratory | Coordinates: (20, 40)
Object: Silverlab (N009) | Type: laboratory | Coordinates: (95, 40)
Object: Cisco Networking Academy Laboratory (N010) | Type: laboratory | Coordinates: (120, 40)
Object: IPSR Lab (N011) | Type: laboratory | Coordinates: (145, 40)
Object: Lab N012 (N012) | Type: laboratory | Coordinates: (170, 40)
Object: Female Toilet (NGT1) | Type: toilet | Coordinates: (195, 40)
Object: Male Toilet (NGT2) | Type: toilet | Coordinates: (205, 40)
Object: Disabled Toilet (NGT3) | Type: toilet | Coordinates: (45, 40)
Object: Male Toilet (NGT4) | Type: toilet | Coordinates: (70, 40)
Object: Female Toilet (NGT5) | Type: toilet | Coordinates: (80, 40)

```

Figure B.4.1: The Output of Successfully Generating The UTAR FICT Classroom Layout

```

=====
CLASSROOM & NEIGHBORS LAYOUT
=====
(N008, Beside Left: None, Beside Right: Disabled Toilet, Opposite: Open-Office Style Classroom)
(NGT3, Beside Left: Microsoft Software Engineering Laboratory, Beside Right: Male Toilet, Opposite: Classroom N006)
(NGT4, Beside Left: Disabled Toilet, Beside Right: Female Toilet, Opposite: Classroom N006)
(NGT5, Beside Left: Male Toilet, Beside Right: Silverlab, Opposite: Classroom N006)
(N009, Beside Left: Female Toilet, Beside Right: Cisco Networking Academy Laboratory, Opposite: Classroom N005)
(N010, Beside Left: Silverlab, Beside Right: Bottom Junction Corridor, Opposite: Classroom N004)
(N011, Beside Left: Bottom Junction Corridor, Beside Right: Lab N012, Opposite: Classroom N003)
(N012, Beside Left: IPSR Lab, Beside Right: Female Toilet, Opposite: Classroom N002)
(NGT1, Beside Left: Male Toilet, Beside Right: Right Corridor, Opposite: Classroom N001)
(NGT2, Beside Left: Lab N012, Beside Right: Female Toilet, Opposite: Classroom N001)
(N007, Beside Left: Left Corridor, Beside Right: Classroom N006, Opposite: Microsoft Software Engineering Laboratory)
(N006, Beside Left: Open-Office Style Classroom, Beside Right: Classroom N005, Opposite: Disabled Toilet, Male Toilet, Female Toilet)
(N005, Beside Left: Classroom N006, Beside Right: Classroom N004, Opposite: Silverlab)
(N004, Beside Left: Classroom N005, Beside Right: Top Junction Corridor, Opposite: Cisco Networking Academy Laboratory)
(N003, Beside Left: Top Junction Corridor, Beside Right: Classroom N002, Opposite: IPSR Lab)
(N002, Beside Left: Classroom N003, Beside Right: Classroom N001, Opposite: Lab N012)
(N001, Beside Left: Classroom N002, Beside Right: None, Opposite: Female Toilet, Male Toilet)

```

Figure B.4.2: The Adjacent and Neighbouring Objects for Each Object.

```

=====
A* routes from N001 to all classrooms, laboratories, and toilets
=====
Dest ID Type Found Distance (m) Path
-----
N002 classroom yes 25.0 N001 → N002
N003 classroom yes 50.0 N001 → N002 → N003
N004 classroom yes 75.0 N001 → N002 → N003 → N004
N005 classroom yes 100.0 N001 → N002 → N003 → N004 → N005
N006 classroom yes 125.0 N001 → N002 → N003 → N004 → N005 → N006
N007 classroom yes 150.0 N001 → N002 → N003 → N004 → N005 → N006 → N007
N008 laboratory yes 180.0 N001 → RIGHT_CORRIDOR → N012 → N011 → N010 → N009 → NGT5 → NGT4 → NGT3 → N008
N009 laboratory yes 105.0 N001 → RIGHT_CORRIDOR → N012 → N011 → N010 → N009
N010 laboratory yes 80.0 N001 → RIGHT_CORRIDOR → N012 → N011 → N010
N011 laboratory yes 55.0 N001 → RIGHT_CORRIDOR → N012 → N011
N012 laboratory yes 30.0 N001 → RIGHT_CORRIDOR → N012
NGT1 toilet yes 45.0 N001 → RIGHT_CORRIDOR → NGT2 → NGT1
NGT2 toilet yes 35.0 N001 → RIGHT_CORRIDOR → NGT2
NGT3 toilet yes 155.0 N001 → RIGHT_CORRIDOR → N012 → N011 → N010 → N009 → NGT5 → NGT4 → NGT3
NGT4 toilet yes 130.0 N001 → RIGHT_CORRIDOR → N012 → N011 → N010 → N009 → NGT5 → NGT4
NGT5 toilet yes 120.0 N001 → RIGHT_CORRIDOR → N012 → N011 → N010 → N009 → NGT5
=====
16/16 routes found.

```

Figure B.4.3: Pathfinding Result of A* from N001 to All Available Destinations

APPENDIX B

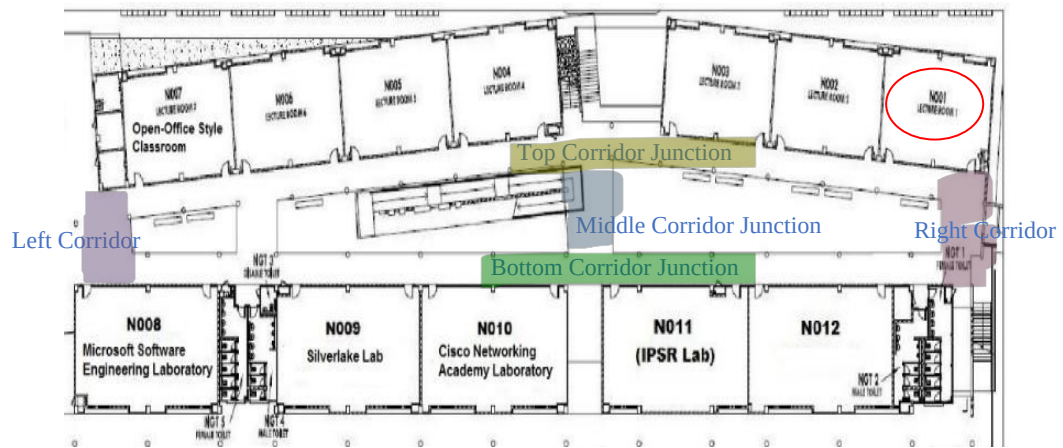


Figure B.4.4: A Portion of The UTAR FICT Ground Floor Map with The Starting Point at N001

APPENDIX C

PLAGIARISM CHECK RESULT

18% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Small Matches (less than 9 words)

Match Groups

-  **112 Not Cited or Quoted** 17%
Matches with neither in-text citation nor quotation marks
-  **4 Missing Quotations** 0%
Matches that are still very similar to source material
-  **6 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 14%  Internet sources
- 2%  Publications
- 16%  Submitted works (Student Papers)

Match Groups

-  **112** Not Cited or Quoted 17%
Matches with neither in-text citation nor quotation marks
-  **4** Missing Quotations 0%
Matches that are still very similar to source material
-  **6** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 14%  Internet sources
- 2%  Publications
- 16%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	Universiti Tunku Abdul Rahman on 2023-09-11	7%
2	Student papers	Universiti Tunku Abdul Rahman on 2024-08-21	1%
3	Internet	eprints.utar.edu.my	1%
4	Student papers	Universiti Tunku Abdul Rahman on 2023-09-07	<1%
5	Student papers	Universiti Tunku Abdul Rahman on 2024-09-24	<1%
6	Internet	orovillerelicensing.water.ca.gov	<1%
7	Internet	www.bomberbot.com	<1%
8	Internet	uokerbala.edu.iq	<1%
9	Student papers	University of Nottingham on 2025-05-13	<1%
10	Student papers	Universiti Tunku Abdul Rahman on 2023-09-11	<1%

APPENDIX C

11	Student papers	Houston Community College on 2025-06-29	<1%
12	Student papers	Universiti Tunku Abdul Rahman on 2024-08-29	<1%
13	Student papers	University of Adelaide on 2025-07-07	<1%
14	Internet	media.readthedocs.org	<1%
15	Internet	github.com	<1%
16	Student papers	CSU, Chico on 2024-05-10	<1%
17	Student papers	University of Queensland on 2024-05-12	<1%
18	Internet	www.mdpi.com	<1%
19	Student papers	Universiti Tunku Abdul Rahman on 2024-08-29	<1%
20	Internet	diffpdf.appspot.com	<1%
21	Internet	www.research4cap.org	<1%
22	Student papers	National University of Ireland, Maynooth on 2024-05-10	<1%
23	Publication	Poonam Nandal, Mamta Dahiya, Meeta Singh, Arvind Dagur, Brijesh Kumar. "Pro...	<1%
24	Student papers	Universidad Internacional de la Rioja on 2025-06-03	<1%

APPENDIX C

25	Internet	pnt.gov	<1%
26	Student papers	Malta College of Arts,Science and Technology on 2025-05-19	<1%
27	Student papers	University of Sunderland on 2024-06-21	<1%
28	Student papers	University of Nottingham on 2022-05-09	<1%
29	Internet	technodocbox.com	<1%
30	Internet	skillapp.co	<1%
31	Student papers	Universiti Tunku Abdul Rahman on 2024-08-20	<1%
32	Internet	dspy.ai	<1%
33	Internet	www.degruyter.com	<1%
34	Student papers	University of Southern Queensland on 2019-10-17	<1%
35	Student papers	University of Sydney on 2024-03-16	<1%
36	Publication	Ton Duc Thang University	<1%
37	Student papers	Dublin City University on 2025-08-03	<1%
38	Internet	blog.17baishi.com	<1%

APPENDIX C

39	Internet	repository.najah.edu	<1%
40	Publication	"Software Engineering Perspectives in Intelligent Systems", Springer Science and ...	<1%
41	Student papers	Mahidol University on 2009-11-04	<1%
42	Student papers	National School of Business Management NSBM, Sri Lanka on 2025-05-09	<1%
43	Student papers	Universiti Tunku Abdul Rahman on 2013-04-08	<1%
44	Student papers	University of London Worldwide on 2024-09-13	<1%
45	Internet	mdpi-res.com	<1%
46	Internet	openjicareport.jica.go.jp	<1%
47	Publication	Arzu Gorgulu Kakisim, Zeynep Turgut. "Multi-channel convolutional neural netwo...	<1%
48	Student papers	October University for Modern Sciences and Arts (MSA) on 2025-06-22	<1%
49	Internet	research-repository.griffith.edu.au	<1%
50	Student papers	Edith Cowan University on 2022-10-28	<1%
51	Publication	Surbhi Gupta, Sangeeta R., Ravi Shankar Mishra, Gaurav Singal, Tapas Badal, Dee...	<1%
52	Student papers	Tilburg University on 2025-05-19	<1%

APPENDIX C

53	Student papers	University of Birmingham on 2025-04-17	<1%
54	Student papers	University of Nottingham on 2025-05-09	<1%
55	Student papers	University of Sydney on 2021-11-09	<1%
56	Student papers	University of Wales, Lampeter on 2024-01-01	<1%
57	Internet	keylabs.ai	<1%
58	Internet	researchrepository.murdoch.edu.au	<1%
59	Internet	smartlogix.co.in	<1%
60	Student papers	Asia Pacific Institute of Information Technology on 2025-06-16	<1%
61	Student papers	Bournemouth University on 2025-05-16	<1%
62	Publication	Bui Thanh Hung, M. Sekar, Ayhan Esi, R. Senthil Kumar. "Applications of Mathema...	<1%
63	Publication	Darius Plikynas, Arunas Zvironas, Marius Gudauskis, Andrius Budrionis, Povilas D...	<1%
64	Student papers	Georgia Institute of Technology Main Campus on 2022-10-07	<1%
65	Publication	Mostafa Elgendy, Cecilia Sik-Lanyi, Arpad Kelemen. "A Novel Marker Detection Sy...	<1%
66	Student papers	New Zealand School of Education on 2025-06-20	<1%

APPENDIX C

67	Student papers	Universiti Teknologi Malaysia on 2018-05-29	<1%
68	Student papers	Universiti Tunku Abdul Rahman on 2023-09-11	<1%
69	Student papers	Universiti Tunku Abdul Rahman on 2024-09-06	<1%
70	Student papers	University of East London on 2023-05-10	<1%
71	Student papers	University of Nottingham on 2018-05-11	<1%
72	Internet	answers.opencv.org	<1%
73	Internet	fuykgy.blogspot.com	<1%
74	Internet	ia601507.us.archive.org	<1%
75	Internet	python.hotexamples.com	<1%
76	Internet	stackoverflow.com	<1%
77	Internet	styjun.blogspot.com	<1%
78	Student papers	universititeknologimara on 2025-07-06	<1%